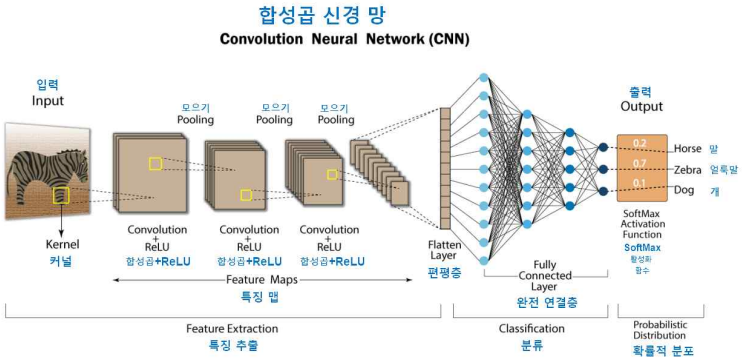
파이썬 인공지능 5

01 CNN의 순전파 이해와 구현

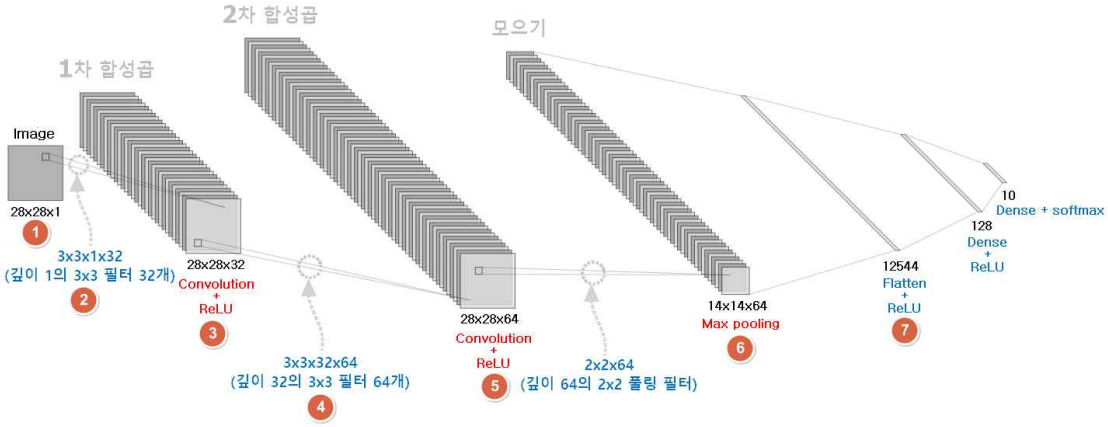
다음은 CNN 형태의 인공 신경망입니다. CNN은 이미지 인식에 뛰어난 인공 신경망으로 이미지의 특징을 뽑아내는 인공 신경망과 분류를 위한 인공 신경망으로 구성됩니다.



이미지의 특징을 뽑아내는 인공 신경망은 전통적인 영상 인식 방법을 접목한 신경망입니다. 전통적인 영상 인식 방법에서는 여러 가지 필터를 이용하여 이미지의 특징을 추출합니다. 필터는 커널이라고도 하며, CNN에서는 학습의 대상이 되는 가중치가 됩니다. 합성 곱(Convolution)은 필터를 이용하여 이미지의 특징을 추출하는 연산을 말하며, 모으기(Pooling)는 합성 곱을 이용하여 얻어낸 이미지를 단순화하는 연산을 말합니다. 분류를 위한 인공 신경망은 앞에서 살펴봤던 신경망으로 완전 연결 층 신경망이라고도 합니다. 여기서는 CNN에서 합성 곱(Convolution)과 모으기(Pooling) 연산의 동작을 살펴보고 구현해 봅니다.

01 CNN 신경망 살펴보기

CNN에서는 입력 이미지가 필터를 거쳐 특징을 가진 이미지로 출력되는 합성 곱(Convolution) 과정, 이미지를 단순화하는 모으기(Pooling) 과정, 앞에서 배웠던 완전 연결 층을 거치는 과정으로 구성됩니다. 예를 들어, 다음은 28x28x1 이미지에 대해 합성 곱 1차, 합성 곱 2차, 모으기, 완전 연결 층으로 구성된 CNN 신경망을 나타냅니다.

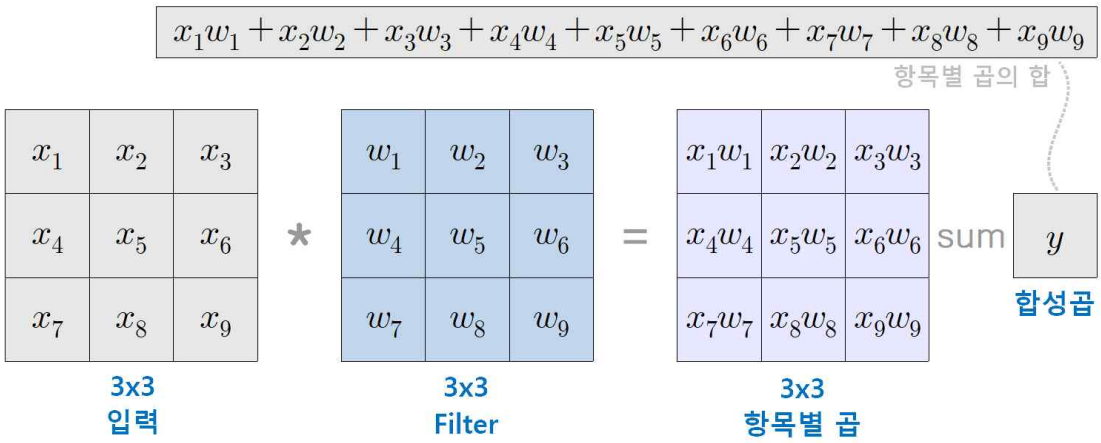


❶ 이 신경망에서 입력 층은 28x28x1의 이미지입니다. CNN에서 입력 이미지는 (세로 픽셀 개수x가로 픽셀 개수x깊이)로 구성됩니다. 여기서는 세로 28픽셀, 가로 28픽셀, 깊이 1의 이미지를 나타냅니다. 깊이는 채널(channel)이라고도 합니다. ❷ 입력 이미지에 필터 1개가 적용되면 특징을 가진 출력 이미지 1개가 나옵니다. 필터의 크기는 1x1, 3x3, 5x5와 같이 같은 크기의 홀수의 곱으로 표현됩니다. 일반적으로 3x3 크기의 필터가 사용됩니다. 입력 이미지에 필터가 적용될 때 필터의 깊이는 반드시 입력 이미지의 깊이와 같아야 합니다. 즉, 합성 곱에 사용하는 필터의 깊이는 입력 이미지의 깊이와 같아야 합니다. 예를 들어, 여기서 사용되는 필터는 3x3x1 크기의 필터가 되어야 합니다. 위 그림에서 28x28x1 이미지는 3x3x1 필터와 합성 곱이 수행된 후, 28x28x1의 특징을 가진 출력 이미지가 됩니다. 즉, 세로 28픽셀, 가로 28픽셀, 깊이 1의 특징을 가진 이미지가 출력됩니다. ❸ 또, 필터의 개수는 출력 이미지의 깊이와 같아야 합니다. 즉, 1차 합성 곱을 거친 출력 이미지의 깊이가 32이므로 1차 합성 곱에 필요한 필터의 개수는 32개가 되어야 합니다. 그래서 1차 합성 곱에 필요한 필터는 3x3x1x32의 크기가 됩니다. 1차 합성 곱을 거친 28x28x32 이미지는 세로 28픽셀, 가로 28픽셀, 깊이 32인 하나의 특징 이미지가 됩니다. ❹ 2차 합성 곱에서도 입력 이미지에 필터 1개가 적용되면 특징을 가진 출력 이미지 1개가 나옵니다. 입력 이미지의 깊이가 32이므로 필터의 크기는 3x3x32가 되어야 합니다. ❺ 2차 합성 곱을 거친 출력 이미지의 깊이가 64이므로 2차 합성 곱에 필요한 필터의 개수는 64개가 되어야 합니다. 그래서 2차 합성 곱에 필요한 필터는 3x3x32x64의 크기가 됩니다. 2차 합성 곱을 거친 28x28x64 이미지는 세로 28픽셀, 가로 28픽셀, 깊이 64인 하나의 특징 이미지가 됩니다. ❻ 모으기 과정은 입력 이미지의 가로, 세로 크기를 줄이는 과정으로 일반적으로 가로, 세로 크기를 각각 1/2로 줄이게 됩니다. 그래서 가로 14, 세로 14, 깊이 64가 됩니다. ❼ Flatten 과정에서는 모으기 과정을 거친 입체 이미지를 1차원 형태로 변형하는 작업을 수행합니다. 즉, 모으기 과정의 입체 이미지를 구성하는 모든 픽셀은 1차원 형태로 나열됩니다. 그래서 14x14x64 = 12544가 됩니다. 이후의 구성은 앞에서 배웠던 신경망의 구성과 같습니다.

02 3x3 입력 : filter size

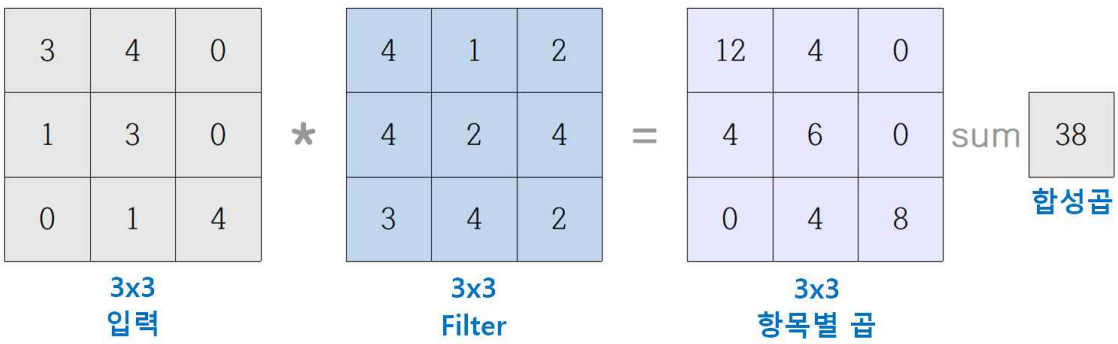
여기서는 3x3 입력 이미지와 3x3 필터에 대해 합성 곱이 수행되는 과정을 살펴봅니다. 이미지의 깊이는 1로 설명의 편의상 생략합니다. 깊이가 고려되는 과정은 뒤에서 구체적으로 살펴봅니다.

다음은 3x3 입력 이미지와 3x3 필터에 대해 합성 곱이 수행되는 과정을 보여줍니다.



3x3 입력 이미지와 3x3 필터는 각각의 대응되는 항목이 개별적으로 곱해져 항목별 곱을 얻은 후에 9개의 항목이 모두 더해져 최종 결과 값 하나를 얻게 됩니다. 합성 곱을 수행할 때 입력의 크기와 필터의 크기는 같아야 합니다. 일반적으로 필터의 크기는 1x1, 3x3, 5x5와 같이 홀수의 곱을 사용합니다.

다음은 3x3 입력 이미지에 대한 합성 곱의 구체적인 예를 보여줍니다.



이 과정을 예제를 통해 살펴봅니다.

1. 다음과 같이 예제를 작성합니다.
512_1.py

```
01 import numpy as np
02
03 np.random.seed(1)
```

```
04
05 image = np.random.randint(5, size=(3,3))
06 print('image =\n', image)
07
08 filter = np.random.randint(5, size=(3,3))
09 print('filter =\n', filter)
10
11 image_x_filter = image * filter
12 print('image_x_filter =\n', image_x_filter)
13
14 convolution = np.sum(image_x_filter)
15 print('convolution =\n', convolution)
```

01 : numpy 모듈을 np라는 이름으로 불러옵니다. numpy 모듈은 행렬이나 다차원 배열을 쉽게 처리 할 수 있도록 지원하는 파이썬의 라이브러리입니다.
03 : np.random.seed 함수를 호출하여 임의 숫자 생성기 모듈을 초기화합니다. 인자로 1을 주어 생성되는 숫자를 일정하게 합니다. 숫자를 변경하면 생성되는 숫자가 달라질 수 있습니다.
05 : 0이상 5미만의 정수를 가진 3x3 행렬을 생성하여 image 변수에 할당합니다.
06 : print 함수를 호출하여 image 변수가 가리키는 행렬 값을 출력해 봅니다.
08 : 0이상 5미만의 정수를 가진 3x3 행렬을 생성하여 filter 변수에 할당합니다.
09 : print 함수를 호출하여 filter 변수가 가리키는 행렬 값을 출력해 봅니다.
11 : image와 filter를 곱한 후, 결과 행렬을 image_x_filter에 할당합니다. 파이썬에서 같은 크기를 갖는 두 개의 NumPy 행렬을 곱하면 같은 위치에 있는 항목끼리 곱셈이 수행됩니다.
12 : print 함수를 호출하여 image_x_filter 변수가 가리키는 행렬 값을 출력해 봅니다.
14 : np.sum 함수를 호출하여 image_x_filter 행렬의 모든 항목 값을 더하여 convolution 변수에 할당합니다.
15 : print 함수를 호출하여 convolution 값을 출력해 봅니다.

2. ▶ 버튼을 눌러 프로그램을 실행시킵니다. 다음은 실행 결과 화면입니다.

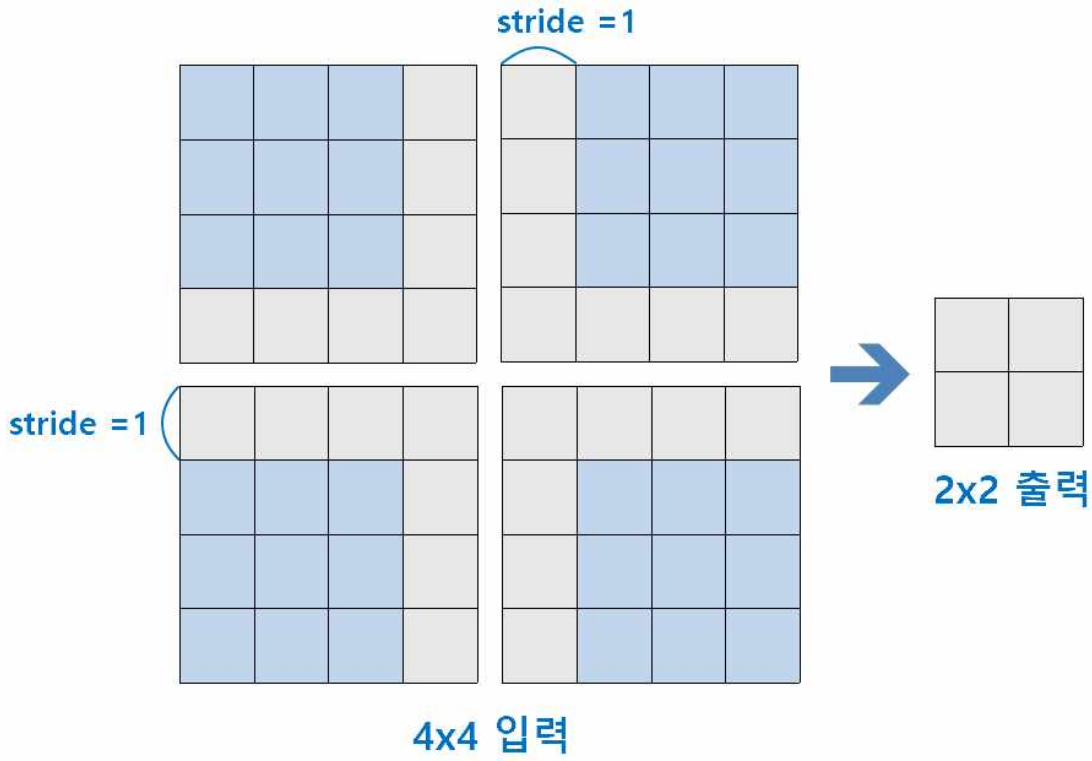


출력 결과를 앞의 그림과 비교하면서 살펴봅니다.

*** 독자의 이해를 돕기 위해 결과 화면을 편집하였습니다.

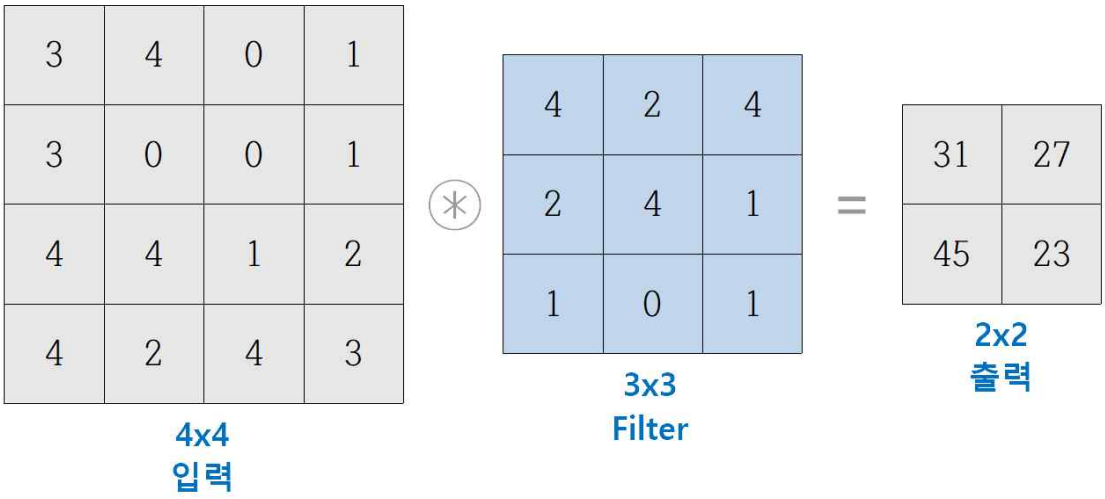
03 4x4 입력 : stride

여기서는 4x4 입력 이미지와 3x3 필터에 대해 합성 곱이 수행되는 과정을 살펴봅니다. 다음은 4x4 입력 이미지와 3x3 필터에 대해 합성 곱이 수행되는 과정을 보여줍니다.

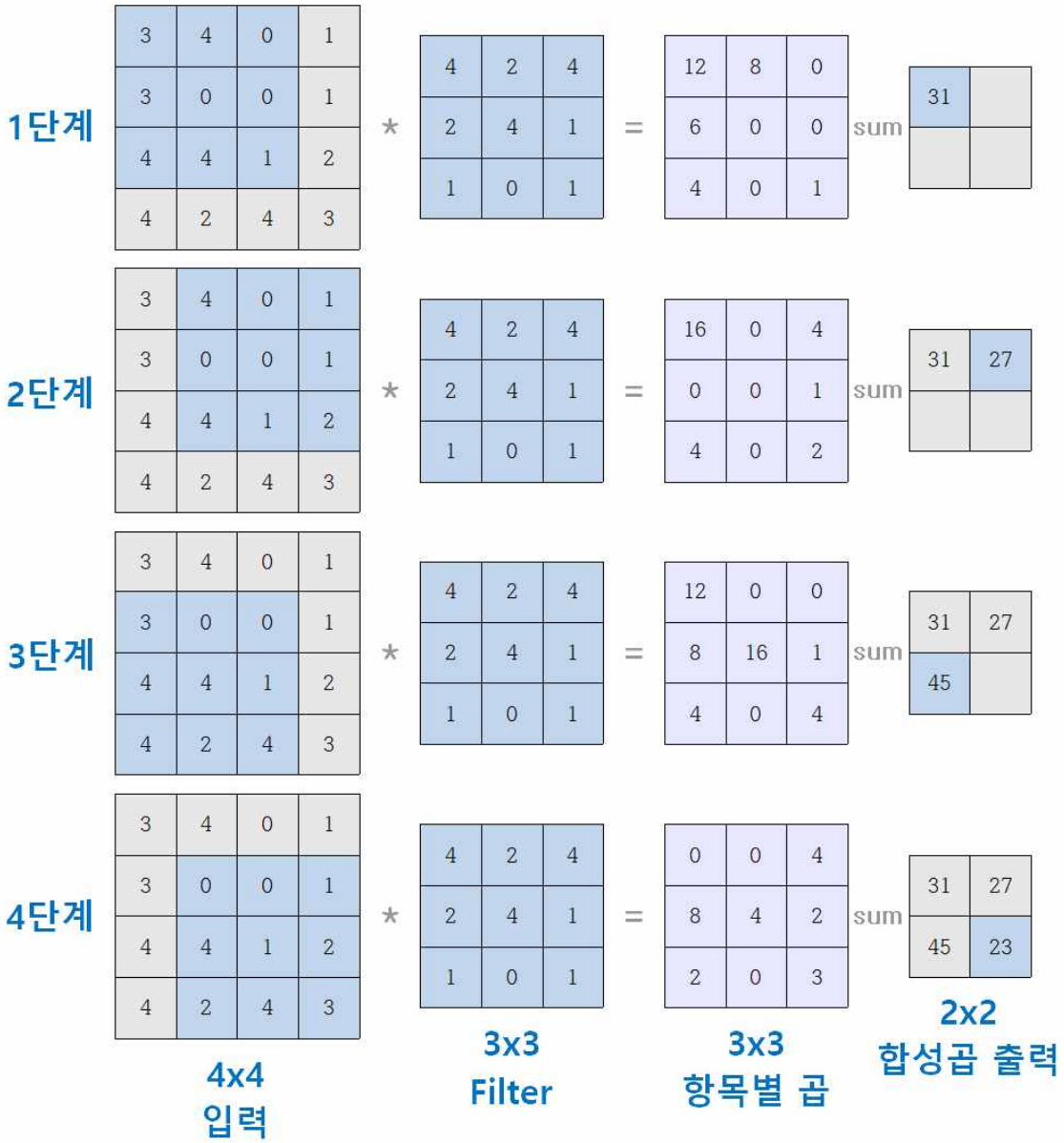


일반적으로 입력 이미지는 필터의 크기보다 큰데, 이때는 필터를 입력 이미지 위에서 일정 간격만큼 왼쪽에서 오른쪽으로 위에서 아래로 옮겨 다니면서 합성 곱을 수행합니다. 이 때 옮겨 다니는 크기를 stride(걸음)라고 합니다. stride 값은 1 이상의 정수 값이 되며 여기서는 1을 사용하고 있습니다. 위 그림은 4x4 입력 이미지로 4개의 3x3 하위 이미지를 생성한 후, 각각의 이미지에 대해 3x3 필터와 합성 곱을 수행하여 2x2 출력 이미지를 생성하는 과정을 나타냅니다.

다음은 4x4 입력 이미지에 대한 합성 곱의 구체적인 예를 보여줍니다.



구체적으로 다음과 같은 과정을 거쳐 합성 곱을 수행합니다.



이 과정을 예제를 통해 살펴봅니다.

1. 다음과 같이 예제를 작성합니다.

513_1.py

```
01 import numpy as np
02
03 np.random.seed(1)
04
05 image = np.random.randint(5, size=(4,4))
06 print('image =\n', image)
```

```
07
08 filter = np.random.randint(5, size=(3,3))
09 print('filter =\n', filter)
10
11 convolution = np.zeros((2,2))
12
13 for row in range(2):
14       for col in range(2):
15             window = image[row:row+3, col:col+3]
16             print('window(%d, %d) =\n' %(row, col), window)
17             print('window(%d, %d)*filter =\n' %(row, col), window*filter)
18             convolution[row, col] = np.sum(window*filter)
19
20 print('convolution =\n', convolution)
```

05 : 0이상 5미만의 임의 정수를 가진 4x4 행렬을 생성하여 image 변수에 할당합니다.

11 : np.zeros 함수를 호출하여 0값으로 채워진 2x2 행렬을 생성하여 convolution 변수에 할당합니다.

13 : row값을 0에서 2 미만까지 바꾸어가며 14~18줄을 2회 수행합니다.

14 : col값을 0에서 2 미만까지 바꾸어가며 15~18줄을 2회 수행합니다.

15 : image 행렬의 row 이상 (row+3) 미만까지, col 이상 (col+3) 미만까지의 행렬을 window 변수가 가리키게 합니다. 예를 들어, (row=0, col=0)일 경우 window 변수는 image[0:3, 0:3] 행렬을 가리키게 됩니다.

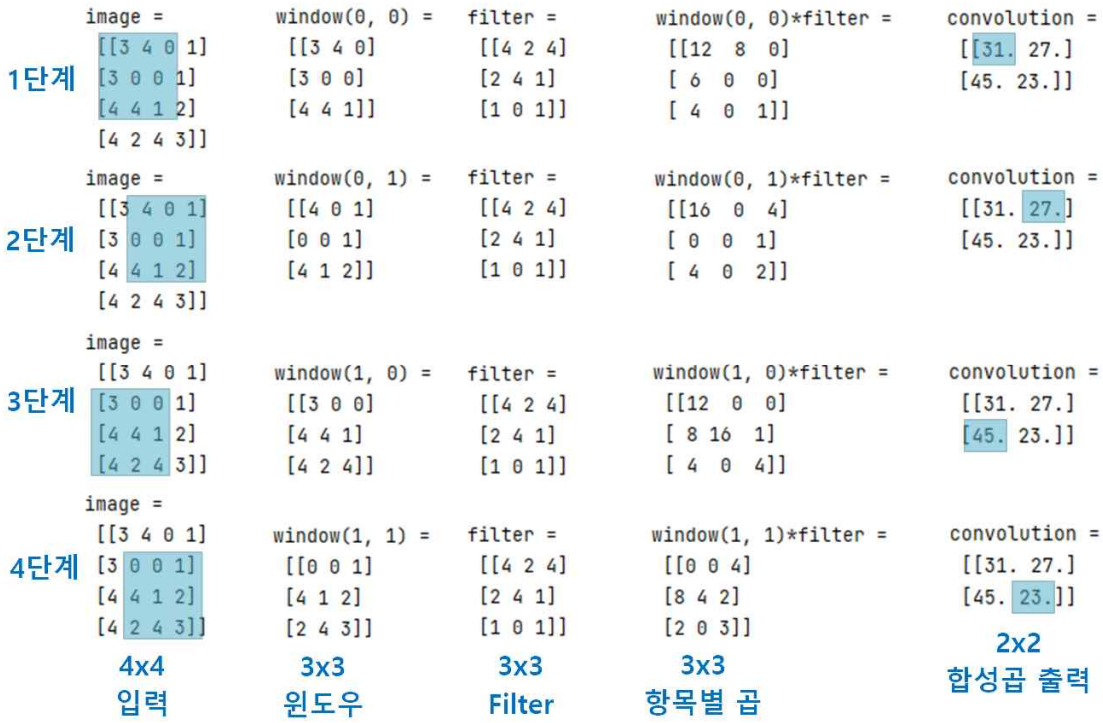
16 : print 함수를 호출하여 row, col값과 window가 가리키는 행렬 값을 출력합니다.

17 : print 함수를 호출하여 row, col값과 window*filter 행렬 값을 출력합니다.

18 : np.sum 함수를 호출하여 window*filter 행렬의 모든 항목의 값을 더해서 convolution 행렬의 (row, col) 항목에 할당합니다.

20 : print 함수를 호출하여 convolution 값을 출력합니다.

2.  버튼을 눌러 프로그램을 실행시킵니다. 다음은 실행 결과 화면입니다.



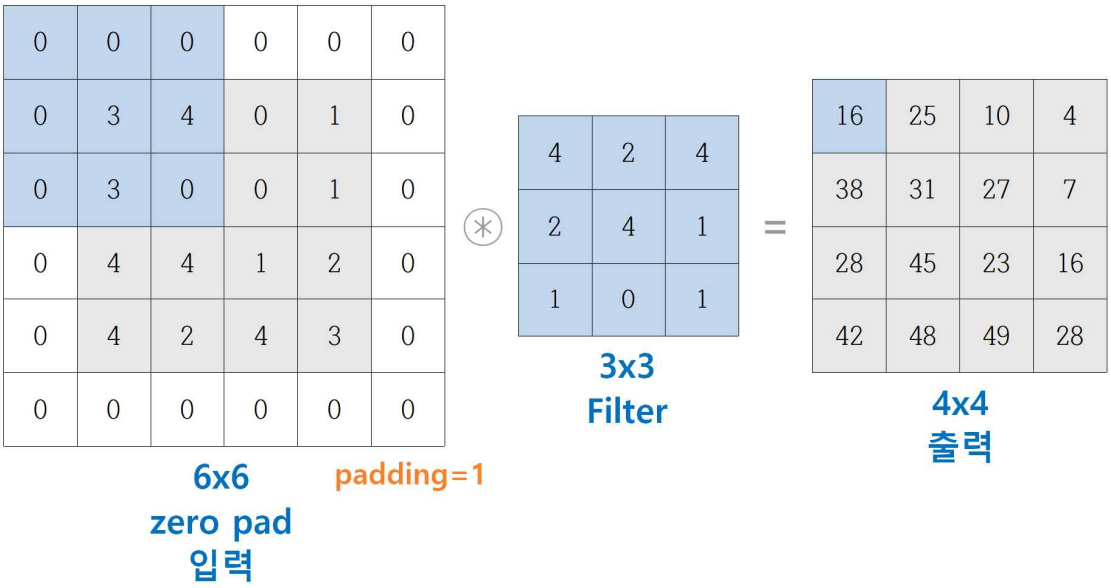
출력 결과를 앞의 그림과 비교하면서 살펴봅니다.

*** 독자의 이해를 돕기 위해 결과 화면을 편집하였습니다.

04 6x6 입력 : padding

앞에서 우리는 4x4 입력 이미지에 대해 3x3 필터와 합성 곱을 수행해 보았습니다. 그 결과 2x2 크기의 출력 이미지를 얻게 되었습니다. 합성 곱을 수행하면 출력 이미지는 입력 이미지보다 작아지게 됩니다. 반복적으로 합성 곱을 수행하면 이미지의 크기는 점점 더 작아지게 됩니다. 입력 이미지의 크기와 출력 이미지의 크기를 같게 하기 위해서는 입력 이미지의 크기를 늘려 주어야 합니다. 이 과정을 padding(붙이기)라고 합니다.

다음은 4x4 입력 이미지를 상하 좌우로 1칸씩 zero padding을 하여 6x6 이미지로 변경한 후, 합성 곱을 수행한 결과를 보여줍니다.



여기서는 padding=1, stride=1로 수행할 경우 출력 이미지의 크기는 입력 이미지의 크기와 같게 됩니다. 일반적으로 입력 이미지의 크기가 N*N, 필터의 크기가 K*K, padding의 크기가 P, stride의 크기가 S라면 출력 이미지의 크기는 다음과 같습니다.

$$\left\lfloor \frac{N+2P-K}{S}+1 \right\rfloor \times \left\lfloor \frac{N+2P-K}{S}+1 \right\rfloor$$

예를 들어, 이미지의 크기가 4x4, 필터의 크기가 3x3, padding의 크기가 1, stride의 크기가 1이라면 출력 이미지의 크기는 다음과 같습니다.

$$\left\lfloor \frac{4+2\times1-3}{1}+1 \right\rfloor \times \left\lfloor \frac{4+2\times1-3}{1}+1 \right\rfloor = 4\times4$$

위의 과정을 예제를 통해 살펴봅니다.

1. 다음과 같이 예제를 작성합니다.

514_1.py

```
01 import numpy as np
02
03 np.random.seed(1)
```

```
04
05 image = np.random.randint(5, size=(4,4))
06 print('image =\n', image)
07
08 filter = np.random.randint(5, size=(3,3))
09 print('filter =\n', filter)
10
11 image_pad = np.pad(image,((1,1), (1,1)))
12 print('image_pad =\n', image_pad)
13
14 convolution = np.zeros((4,4))
15
16 for row in range(4):
17       for col in range(4):
18                window = image_pad[row:row+3, col:col+3]
19                convolution[row, col] = np.sum(window*filter)
20
21 print('convolution =\n', convolution)
```

11 : np.pad 함수를 호출하여 image 행렬과 같은 모양의 행렬을 내부적으로 생성한 후, 행을 상하로 한 칸씩, 열을 좌우로 한 칸씩 늘려 0으로 채운 후, image_pad 변수에 할당합니다.

12 : print 함수를 호출하여 image_pad 변수가 가리키는 행렬 값을 출력해 봅니다.

14 : np.zeros 함수를 호출하여 0값으로 채워진 4x4 행렬을 생성하여 convolution 변수에 할당합니다.

16 : row값을 0에서 4 미만까지 바꾸어가며 17~19줄을 4회 수행합니다.

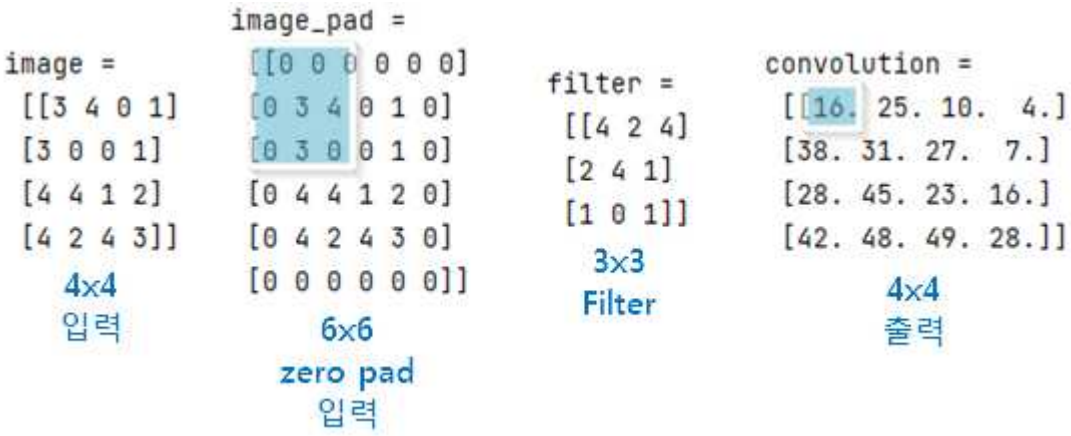
17 : col값을 0에서 4 미만까지 바꾸어가며 18~19줄을 4회 수행합니다.

18 : image_pad 행렬의 row 이상 (row+3) 미만까지, col 이상 (col+3) 미만까지의 행렬을 window 변수가 가리키게 합니다. 예를 들어, (row=0, col=0)일 경우 window 변수는 image_pad[0:3, 0:3] 행렬을 가리키게 됩니다.

19 : np.sum 함수를 호출하여 window*filter 행렬의 모든 항목의 값을 더해서 convolution 행렬의 (row, col) 항목에 할당합니다.

21 : print 함수를 호출하여 convolution 행렬 값을 출력합니다.

2.  버튼을 눌러 프로그램을 실행시킵니다. 다음은 실행 결과 화면입니다.

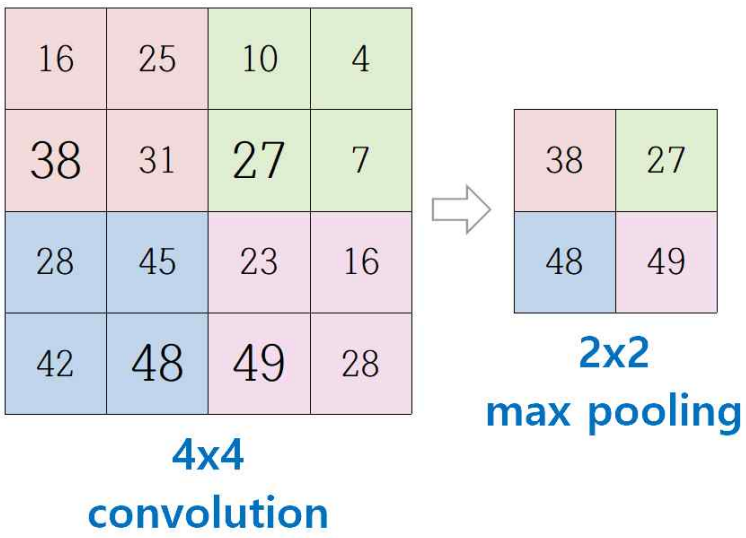


출력 결과를 앞의 그림과 비교하면서 살펴봅니다.

*** 독자의 이해를 돕기 위해 결과 화면을 편집하였습니다.

05 4x4 합성 곱 : pooling

다음은 4x4 합성 곱 이미지에 대해 max pooling을 수행한 결과를 보여줍니다. pooling은 특별한 종류의 필터로 생각할 수 있습니다. max pooling은 pooling 필터가 적용되는 영역에서 최대값을 골라냅니다.



일반적으로 pooling 필터의 크기는 2x2가 사용되며, stride는 2가 됩니다.

이 과정을 예제를 통해 살펴봅니다.

1. 다음과 같이 예제를 작성합니다.

515_1.py

```
01 import numpy as np
02
03 np.random.seed(1)
04
05 image = np.random.randint(5, size=(4,4))
06 print('image =\n', image)
07
08 filter = np.random.randint(5, size=(3,3))
09 print('filter =\n', filter)
10
11 image_pad = np.pad(image,((1,1), (1,1)))
12 print('image_pad =\n', image_pad)
13
14 convolution = np.zeros((4,4))
15
16 for row in range(4):
17     for col in range(4):
18         window = image_pad[row:row+3, col:col+3]
19         convolution[row, col] = np.sum(window*filter)
20
21 print('convolution =\n', convolution)
22
23 max_pooled = np.zeros((2,2))
24
25 for row in range(0, 2):
26     for col in range(0, 2):
27         window = convolution[2*row:2*row+2, 2*col:2*col+2]
28         max_pooled[row, col] = np.max(window)
29
30 print('max_pooled =\n', max_pooled)
```

23 : np.zeros 함수를 호출하여 0값으로 채워진 2x2 행렬을 생성하여 max_pooled 변수에 할당합니다.

25 : row값을 0에서 2 미만까지 26~28줄을 2회 수행합니다.

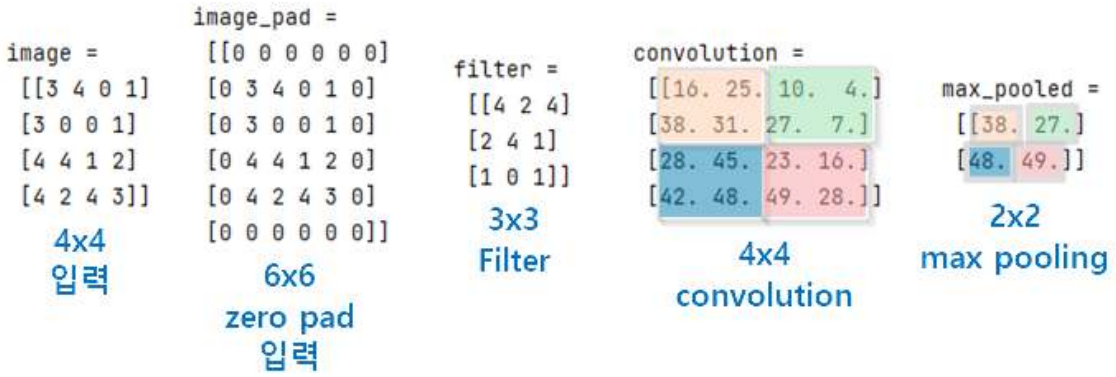
26 : col값을 0에서 2 미만까지 27~28줄을 2회 수행합니다.

27 : convolution 행렬의 2*row 이상 (2*row+2) 미만까지, 2*col 이상 (2*col+2) 미만까지의 행렬을 window 변수가 가리키게 합니다. 예를 들어, (row=0, col=0)일 경우 window 변수는 convolution[0:2, 0:2] 행렬을 가리키게 됩니다.

28 : np.max 함수를 호출하여 window가 가리키는 행렬의 항목 중에 최대값을 갖는 항목을 max_pooled 행렬의 (row, col) 항목에 할당합니다.

30 : print 함수를 호출하여 max_pooled 행렬 값을 출력합니다.

2. ▶ 버튼을 눌러 프로그램을 실행시킵니다. 다음은 실행 결과 화면입니다.

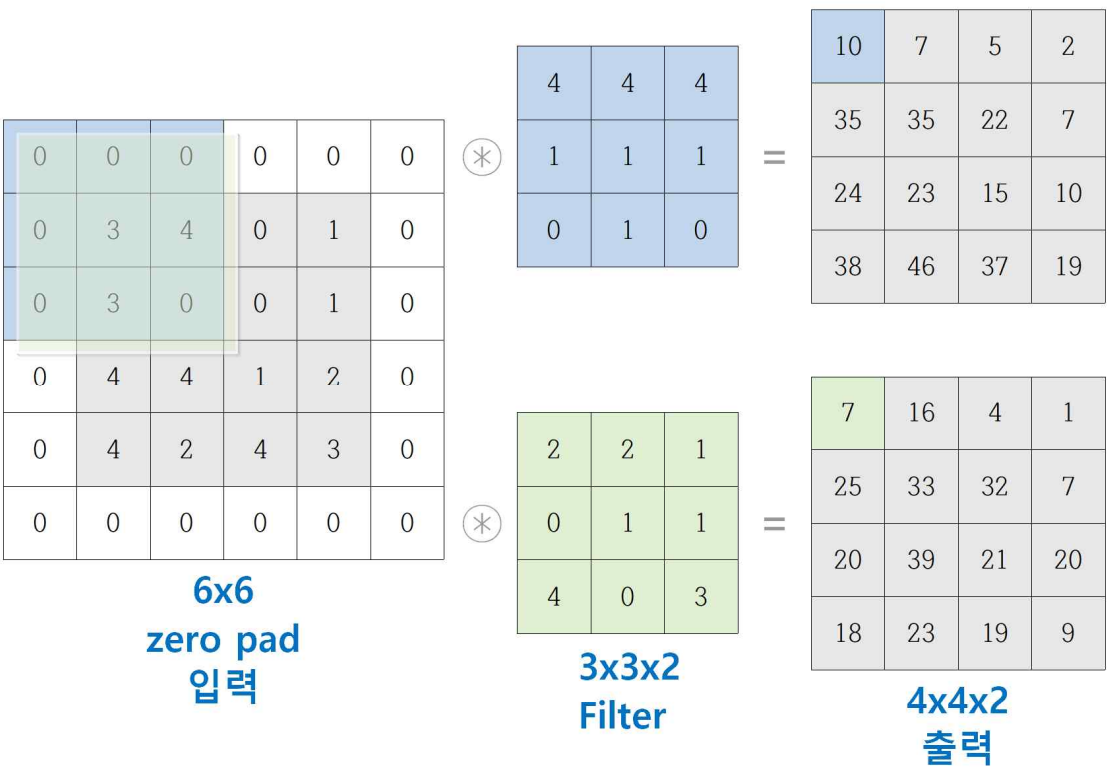


출력 결과를 앞의 그림과 비교하면서 살펴봅니다.

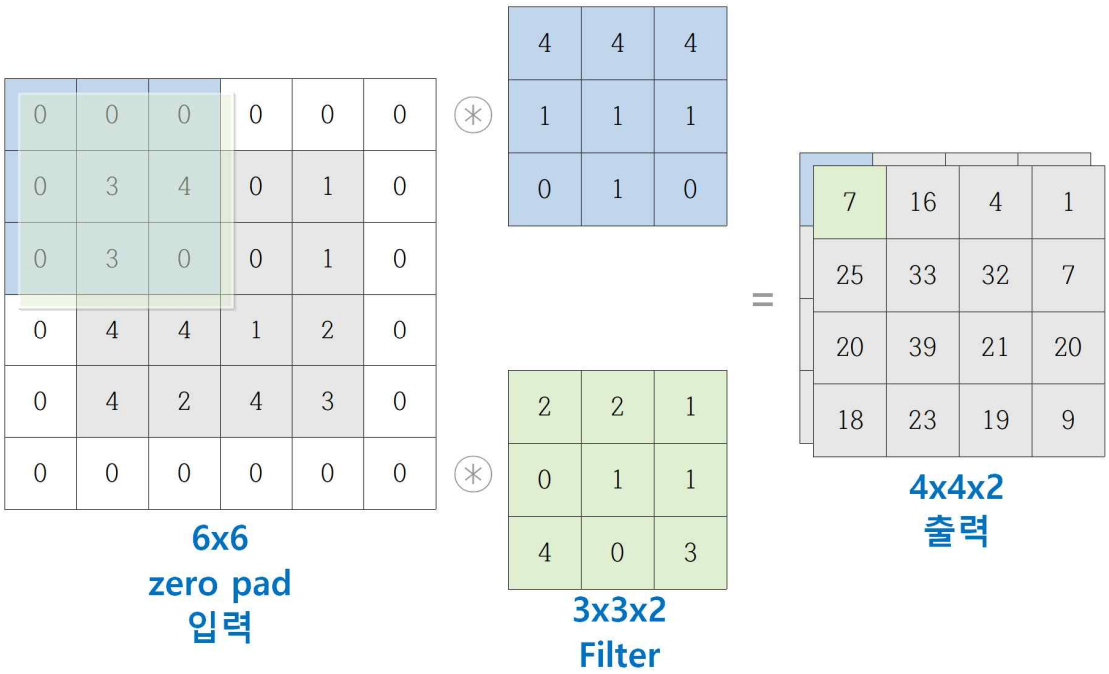
*** 독자의 이해를 돕기 위해 결과 화면을 편집하였습니다.

06 6x6 입력 필터 늘리기

다음은 6x6 입력 이미지에 대해 2개의 필터를 이용하여 2개의 출력 이미지를 얻는 과정을 보여줍니다. 필터는 이미지의 특징을 추출하는 역할을 하므로 여기서는 입력 이미지에서 2가지 특징을 추출하게 됩니다. 일반적으로 필터의 개수만큼 이미지의 특징이 추출됩니다.

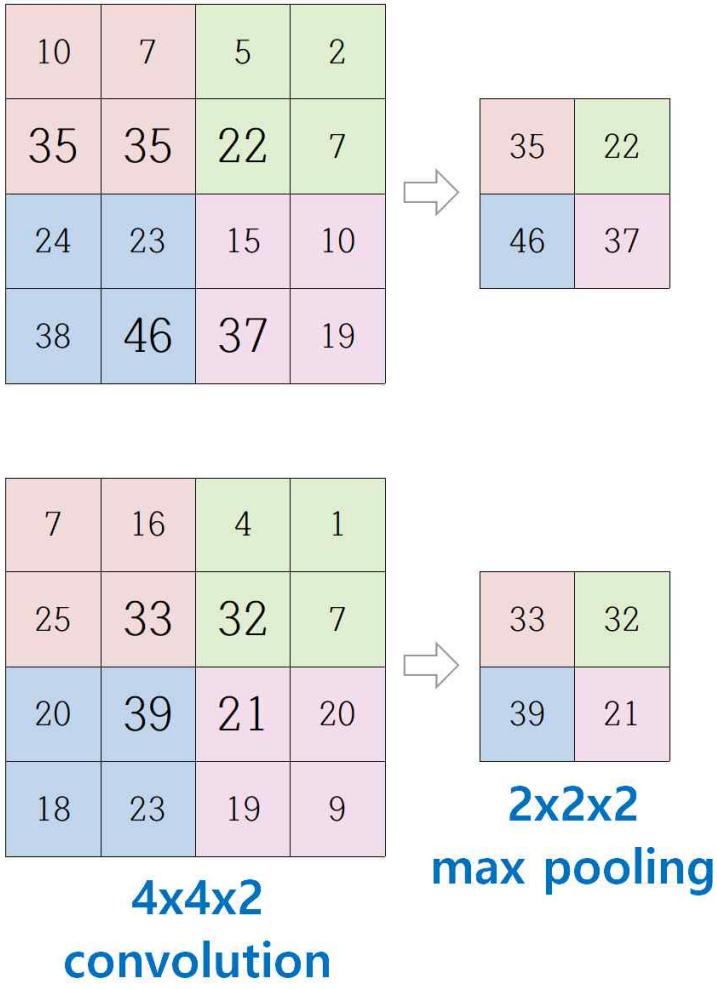


추출된 2장의 이미지는 다음과 같이 겹친 상태로 2층으로 쌓여진 하나의 입체 이미지를 구성합니다. 즉, 겹쳐진 개수만큼의 깊이를 갖는 하나의 이미지를 구성합니다.



일반적으로 필터의 개수만큼 출력 이미지가 생성되며, 출력 이미지는 겹쳐진 상태로 하나의 입체 이미지로 출력됩니다. CNN에서는 최초에 입력된 이미지의 경우도 1층짜리 하나의 입체 이미지로 처리됩니다.

다음은 4x4x2 출력 이미지에 대해 max pooling을 수행한 결과를 보여줍니다.



지금까지의 과정을 예제를 통해 정리해 봅니다.

1. 다음과 같이 예제를 작성합니다.

516_1.py

```
01 import numpy as np
02
03 np.random.seed(1)
04
05 image = np.random.randint(5, size=(4,4))
06 print('image =\n', image)
07
08 filter = np.random.randint(5, size=(3,3,2))
09 print('filter_0 =\n', filter[:,:,0])
10 print('filter_1 =\n', filter[:,:,1])
11
```

```
12 image_pad = np.pad(image,((1,1), (1,1)))
13 print('image_pad =\n', image_pad)
14
15 convolution = np.zeros((4,4,2))
16
17 for fn in range(2):
18     for row in range(4):
19         for col in range(4):
20             window = image_pad[row:row+3, col:col+3]
21             convolution[row, col, fn] = np.sum(window*filter[:,:,fn])
22
23 print('convolution_0 =\n', convolution[:,:,0])
24 print('convolution_1 =\n', convolution[:,:,1])
25
26 max_pooled = np.zeros((2,2,2))
27
28 for fn in range(2):
29     for row in range(0, 2):
30         for col in range(0, 2):
31             window = convolution[2*row:2*row+2, 2*col:2*col+2, fn]
32             max_pooled[row, col, fn] = np.max(window)
33
34 print('max_pooled_0 =\n', max_pooled[:,:,0])
35 print('max_pooled_1 =\n', max_pooled[:,:,1])
```

08 : 0이상 5미만의 정수를 가진 3x3x2 행렬을 생성하여 filter 변수에 할당합니다. 이렇게 하면 3x3 행렬 2개가 생성됩니다. 이 경우에는 3x3 크기의 필터 2개로 해석합니다.

09 : print 함수를 호출하여 첫 번째 filter 행렬 값을 출력해 봅니다.

10 : print 함수를 호출하여 두 번째 filter 행렬 값을 출력해 봅니다.

15 : np.zeros 함수를 호출하여 0값으로 채워진 4x4x2 행렬을 생성하여 convolution 변수에 할당합니다. 이 경우에는 2의 depth(깊이)를 갖는 4x4 행렬로 해석합니다.

17 : fn값을 0에서 2 미만까지 바꾸어가며 18~21줄을 2회 수행합니다. fn은 filter number를 의미합니다.

18 : row값을 0에서 4 미만까지 바꾸어가며 19~21줄을 4회 수행합니다.

19 : col값을 0에서 4 미만까지 바꾸어가며 20~21줄을 4회 수행합니다.

20 : image_pad 행렬의 row 이상 (row+3) 미만까지, col 이상 (col+3) 미만까지의 행렬을 window 변수가 가리키게 합니다. 예를 들어, (row=0, col=0)일 경우 window 변수는 image_pad[0:3, 0:3] 행렬을 가리키게 됩니다.

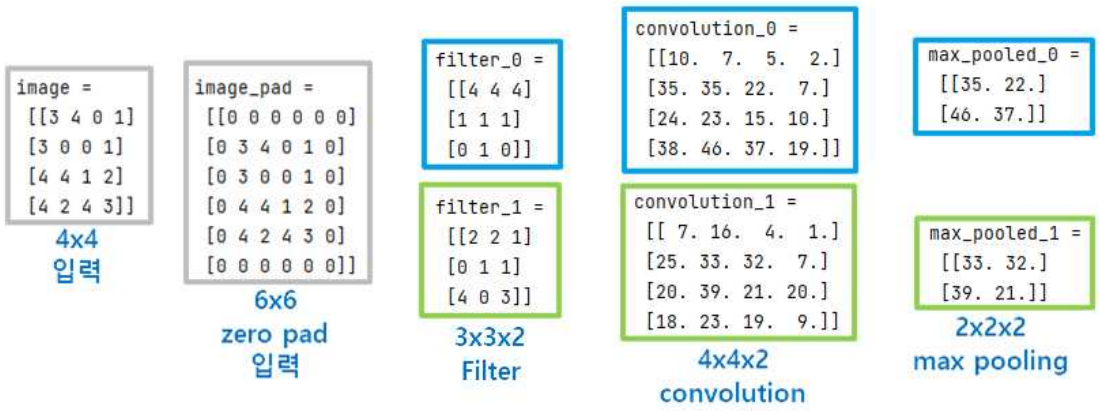
21 : np.sum 함수를 호출하여 window*filter[:,:,fn] 행렬의 모든 항목의 값을 더해서 convolution 행렬의 (row, col, fn) 항목에 할당합니다.

23 : print 함수를 호출하여 convolution[:,:,0] 행렬 값을 출력합니다.

24 : print 함수를 호출하여 convolution[:,:,1] 행렬 값을 출력합니다.

26 : np.zeros 함수를 호출하여 0값으로 채워진 2x2x2 행렬을 생성하여 max_pooled 변수에 할당합니다. 이 경우에는 맨 뒤에 오는 2의 depth(깊이)를 갖는 2x2 행렬로 해석합니다.
28 : fn값을 0에서 2 미만까지 바꾸어가며 29~32줄을 2회 수행합니다.
29 : row값을 0에서 2 미만까지 30~32줄을 2회 수행합니다.
30 : col값을 0에서 2 미만까지 31~32줄을 2회 수행합니다.
31 : convolution 행렬의 [2*row 이상 (2*row+2) 미만, 2*col 이상 (2*col+2) 미만, fn 깊이]에 있는 행렬을 window 변수가 가리키게 합니다. 예를 들어, (row=0, col=0, fn=0)일 경우 window 변수는 convolution[0:2, 0:2, 0] 행렬을 가리키게 됩니다.
32 : np.max 함수를 호출하여 window가 가리키는 행렬의 항목 중에 최대값을 갖는 항목을 max_pooled 행렬의 (row, col, fn) 항목에 할당합니다.
34 : print 함수를 호출하여 max_pooled[:,:,0] 행렬 값을 출력합니다.
35 : print 함수를 호출하여 max_pooled[:,:,1] 행렬 값을 출력합니다.

2. ▶ 버튼을 눌러 프로그램을 실행시킵니다. 다음은 실행 결과 화면입니다.



출력 결과를 앞의 그림과 비교하면서 살펴봅니다.

*** 독자의 이해를 돕기 위해 결과 화면을 편집하였습니다.

07 필터 역할 살펴보기

앞에서 우리는 입력 이미지에 3x3 필터를 적용하여 합성 곱을 수행하는 과정을 살펴보았습니다. 이 과정을 영상 인식에서는 필터링이라고 합니다. 필터링이란 이미지나 영상에서 원하는 정보만 걸러내는 작업을 말합니다. 이 과정에서 잡음을 걸러내어 영상을 깨끗하게 만들거나 잔선을 제거하여 흐리게 만들거나 부드러운 느낌을 걸러내어 선명한 느낌의 영상을 만들 수 있습니다. 필터링은 필터, 커널, 마스크, 윈도우 등으로 불리는 작은 크기의 행렬을 이용합니다. 필터링 연산의 결과는 행렬의 크기와 각 항목의 값에 의해 결정됩니다. 여기서는 필터링을 통해 추출되는 이미지의 특징을 구체적으로 살펴봅니다.

다음은 여러 나라의 문자가 표시된 큐브 이미지입니다. 이 큐브 이미지에 몇 가지 필터를 적

용하여 이미지의 특징을 추출해 봅니다.



부드러운 이미지 추출하기

먼저 평균값 필터를 사용해 봅니다. 평균값 필터는 이웃 화소들을 평균화하여 이미지를 부드럽게 만들어주는 역할을 합니다. 평균값 필터는 다음과 같습니다.

$$K = \frac{1}{9}\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

이 필터는 주변의 모든 픽셀을 더해준 후, 필터의 원소 개수만큼 나누어줍니다.

1. 다음과 같이 예제를 작성합니다.

517_1.py

```
01 import numpy as np
02 import cv2
03 import matplotlib.pyplot as plt
04
05 image_color = cv2.imread('cube.png')
```

```
06 print('image_color.shape =', image_color.shape)
07 image = cv2.cvtColor(image_color, cv2.COLOR_BGR2GRAY)
08 print('image.shape =', image.shape)
09
10 filter = np.array([
11       [1,1,1],
12       [1,1,1],
13       [1,1,1],
14 ])/9
15
16 image_pad = np.pad(image,((1,1), (1,1)))
17 print('image_pad.shape =', image_pad.shape)
18
19 convolution = np.zeros_like(image)
20
21 for row in range(image.shape[0]):
22       for col in range(image.shape[1]):
23             window = image_pad[row:row+3, col:col+3]
24             convolution[row, col] = np.clip(np.sum(window*filter), 0, 255)
25
26 images = [image, convolution]
27 labels = ['gray', 'convolution']
28
29 plt.figure(figsize=(10, 5))
30 for i in range(len(images)):
31       plt.subplot(1,2,i+1)
32       plt.xticks([])
33       plt.yticks([])
34       plt.imshow(images[i], cmap=plt.cm.gray)
35       plt.xlabel(labels[i])
36 plt.show()
```

02 : import문을 이용하여 cv2 모듈을 불러옵니다. cv2는 OpenCV에 대한 파이썬 라이브러리입니다. OpenCV 라이브러리는 영상 처리 라이브러리입니다.

03 : import문을 이용하여 matplotlib.pyplot 모듈을 plt라는 이름으로 불러옵니다. 여기서는 matplotlib.pyplot 모듈을 이용하여 29~36 줄에서 이미지를 출력합니다.

05 : cv2 모듈의 imread 함수를 호출하여 cube.png 파일을 읽어와 image_color 변수로 가리키게 합니다. imread 함수는 numpy.ndarray 객체를 내어줍니다. numpy.ndarray는 numpy 모듈에서 제공하는 배열입니다.

06 : print 함수를 호출하여 image_color의 모양을 출력합니다.

07 : cv2 모듈의 cvtColor 함수를 호출하여 image_color 변수가 가리키는 그림의 색깔을 바꾼 후, 바뀐 그림을 image 변수가 가리키도록 합니다. BGR 형식의 파일을 GRAY 형식의 파

일로 바꿉니다. OpenCV의 색깔 형식을 RGB라고 하나 실제로는 바이트 데이터의 순서가 반대인 BGR 형식입니다.
08 : print 함수를 호출하여 image의 모양을 출력합니다.
10~14 : np.array 함수를 호출하여, 1로 채워진 3x3 행렬을 생성하여 각 항목을 9로 나눠준 후, filter 변수에 할당합니다. 이 필터는 영상 처리에서 사용하는 필터로 평균값 필터라고 하며, 그림을 흐릿하게 하여 잔선을 제거해 주는 역할을 합니다.
16 : np.pad 함수를 호출하여 image 행렬과 같은 모양의 행렬을 내부적으로 생성한 후, 행을 상하로 한 칸씩, 열을 좌우로 한 칸씩 늘려 0으로 채운 후, image_pad 변수에 할당합니다.
17 : print 함수를 호출하여 image_pad의 모양을 출력합니다.
19 : np.zeros_like 함수를 호출하여 0으로 채워진 image 모양과 같은 행렬을 생성합니다.
21 : row값을 0에서 image의 세로 크기 미만까지 바꾸어가며 22~24줄을 수행합니다.
22 : col값을 0에서 image의 가로 크기 미만까지 바꾸어가며 23~24줄을 수행합니다.
23 : image_pad 행렬의 row 이상 (row+3) 미만까지, col 이상 (col+3) 미만까지의 행렬을 window 변수가 가리키게 합니다. 예를 들어, (row=0, col=0)일 경우 window 변수는 image_pad[0:3, 0:3] 행렬을 가리키게 됩니다.
24 : np.sum 함수를 호출하여 window*filter 행렬의 모든 항목의 값을 더하고, np.clip 함수를 이용하여 0에서 255 사이로 제한해 준 후에, convolution 행렬의 (row, col) 항목에 할당합니다.
21 : print 함수를 호출하여 convolution 행렬 값을 출력합니다.
26 : image와 convolution을 리스트에 넣은 후, images 변수에 할당합니다. images는 29~36 줄에서 그릴 이미지 리스트입니다.
27 : 'gray'와 'convolution' 문자열을 리스트에 넣은 후, labels 변수에 할당합니다.
29 : plt.figure 함수를 호출하여 새로운 그림을 만들 준비를 합니다. figure 함수는 내부적으로 그림을 만들고 편집할 수 있게 해 주는 함수입니다. figsize는 그림의 인치 단위의 크기를 나타냅니다. 여기서는 가로 10인치, 세로 5인치의 그림을 그린다는 의미입니다.
30 : 0에서 images 리스트 크기 미만에 대해
31 : plt.subplot 함수를 호출하여 그림 창을 분할하여 하위 그림을 그립니다. 1,2는 각각 행의 개수와 열의 개수를 의미합니다. i+1은 하위 그림의 위치를 나타냅니다.
32, 33 : plt.xticks, plt.yticks 함수를 호출하여 x, y 축 눈금을 설정합니다. 여기서는 빈 리스트를 주어 눈금 표시를 하지 않습니다.
34 : plt.imshow 함수를 호출하여 images[i] 항목의 그림을 내부적으로 그립니다. cmap는 color map의 약자로 gray는 그림을 흑백으로 표현해 줍니다.
35 : plt.xlabel 함수를 호출하여 x 축에 라벨을 붙여줍니다. 라벨의 값은 labels[i]입니다.
36 : plt.show 함수를 호출하여 내부적으로 그린 그림을 화면에 그립니다.

2. ▶ 버튼을 눌러 프로그램을 실행시킵니다. 다음은 실행 결과 화면입니다.

```
image_color.shape = (500, 500, 3)
image.shape = (500, 500)
image_pad.shape = (502, 502)
```

원 이미지의 크기는 500x500x3이며, 흑백으로 처리한 후에는 500x500의 크기가 됩니다. 그리고 패딩을 수행한 후에는 502x502의 크기가 됩니다.


gray


convolution

오른쪽 이미지가 왼쪽 이미지에 비해 조금 흐릿한 것을 확인합니다.

선명한 이미지 추출하기

다음은 Sharpening 필터를 사용해 봅니다. Sharpening 필터는 경계를 선명하게 표현합니다. Sharpening 필터는 다음과 같습니다.

$$K = \begin{bmatrix} -1 & -1 & -1 \\ -1 & 9 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

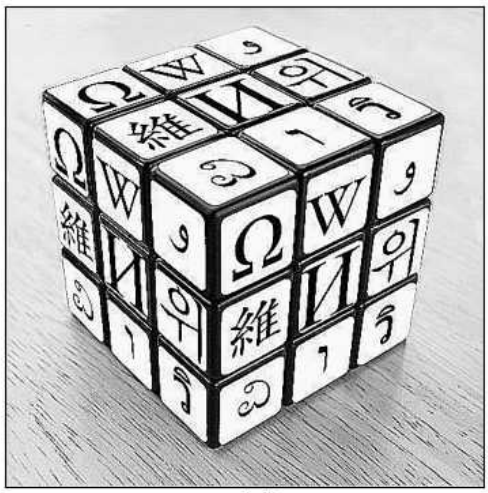
1. 다음과 같이 예제를 수정합니다.

```
10 filter = np.array([
11         [-1,-1,-1],
12         [-1, 9,-1],
13         [-1,-1,-1]
14 ])
```

10~14 : np.array 함수를 호출하여, 예제와 같은 숫자로 채워진 3x3 행렬을 생성한 후, filter 변수에 할당합니다. 이 필터는 영상 처리에서 사용하는 필터로 sharpening 필터라고 하며, 그림을 선명하게 해 주는 역할을 합니다.

2.  버튼을 눌러 프로그램을 실행시킵니다. 다음은 실행 결과 화면입니다.





오른쪽 이미지가 왼쪽 이미지에 비해 경계선이 선명한 것을 확인합니다.

경계선 추출하기

다음은 Edge detection 필터를 사용해 봅니다. Edge detection 필터는 경계선을 추출합니다. Edge detection 필터는 다음과 같습니다.

$$K = \begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

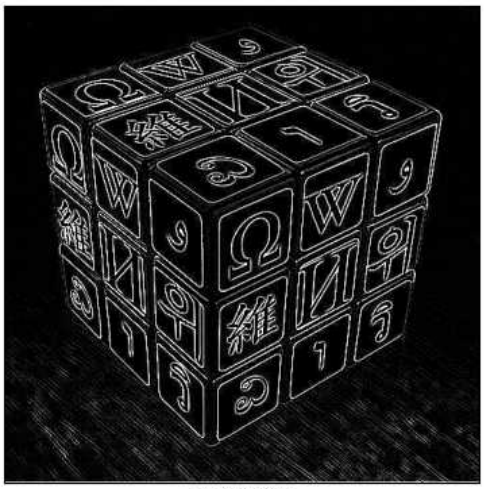
1. 다음과 같이 예제를 수정합니다.

```
10 filter = np.array([
11         [-1,-1,-1],
12         [-1, 8,-1],
13         [-1,-1,-1]
14 ])
```

10~14 : np.array 함수를 호출하여, 예제와 같은 숫자로 채워진 3x3 행렬을 생성한 후, filter 변수에 할당합니다. 이 필터는 영상 처리에서 사용하는 필터로 edge detection 필터라고 하며, 사물의 경계선을 추출해 주는 역할을 합니다.

2. ▶ 버튼을 눌러 프로그램을 실행시킵니다. 다음은 실행 결과 화면입니다.


gray


convolution

왼쪽 이미지에서 경계선이 추출된 것을 확인합니다.

*** 기존의 영상 인식에서는 프로그래머가 필터를 정해 주어야 했으나 CNN에서는 이러한 필터들이 학습을 통해 만들어지게 됩니다. 즉, CNN에서 필터는 학습의 대상이 되는 가중치가 됩니다.

이미지 단순화

모으기(Pooling)는 합성 곱을 이용하여 얻어낸 이미지를 단순화하는 연산을 말합니다.
여기서는 max pooling을 이용하여 이미지를 단순화해 봅니다.

1. 다음과 같이 예제를 수정합니다.

517_2.py

```
01 import numpy as np
02 import cv2
03 import matplotlib.pyplot as plt
04
05 image_color = cv2.imread('cube.png')
06 print('image_color.shape =', image_color.shape)
07 image = cv2.cvtColor(image_color, cv2.COLOR_BGR2GRAY)
08 print('image.shape =', image.shape)
09
10 filter = np.array([
11       [-1,-1,-1],
12       [-1, 8,-1],
```

```
13        [-1,-1,-1]
14 ])
15
16 image_pad = np.pad(image,((1,1), (1,1)))
17 print('image_pad.shape =', image_pad.shape)
18
19 convolution = np.zeros_like(image)
20
21 for row in range(image.shape[0]):
22        for col in range(image.shape[1]):
23                window = image_pad[row:row+3, col:col+3]
24                convolution[row, col] = np.clip(np.sum(window*filter), 0, 255)
25
26 max_pooled = np.zeros((int(image.shape[0]/2),int(image.shape[1]/2)))
27
28 for row in range(0, int(image.shape[0]/2)):
29        for col in range(0, int(image.shape[1]/2)):
30                window = convolution[2*row:2*row+2, 2*col:2*col+2]
31                max_pooled[row, col] = np.max(window)
32
33 images = [image, convolution, max_pooled]
34 labels = ['gray', 'convolution', 'max_pooled']
35
36 plt.figure(figsize=(15, 5))
37 for i in range(len(images)):
38        plt.subplot(1,3,i+1)
39        plt.xticks([])
40        plt.yticks([])
41        plt.imshow(images[i], cmap=plt.cm.gray)
42        plt.xlabel(labels[i])
43 plt.show()
```

26 : np.zeros 함수를 호출하여 0값으로 채워진 (image 가로 크기/2)*(image 세로 크기/2) 행렬을 생성하여 max_pooled 변수에 할당합니다.

28 : row값을 0에서 (image 세로 크기/2) 미만까지 29~31줄을 수행합니다.

29 : col값을 0에서 (image 가로 크기/2) 미만까지 30~31줄을 수행합니다.

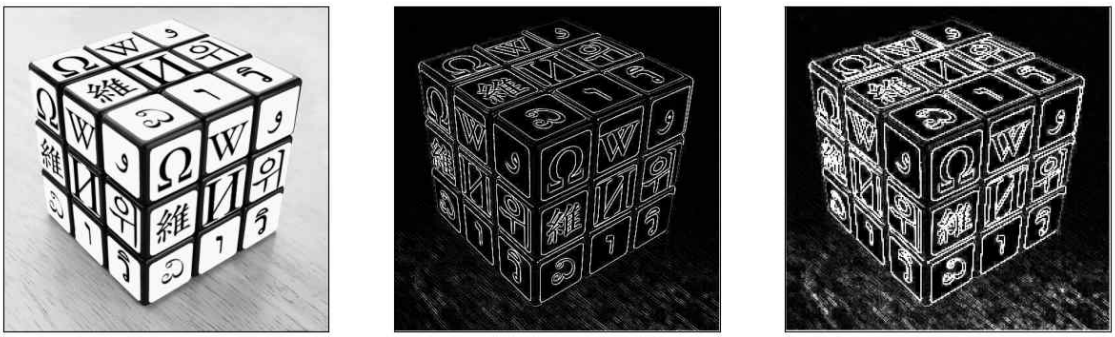
30 : convolution 행렬의 2*row 이상 (2*row+2) 미만까지, 2*col 이상 (2*col+2) 미만까지의 행렬을 window 변수가 가리키게 합니다. 예를 들어, (row=0, col=0)일 경우 window 변수는 convolution[0:2, 0:2] 행렬을 가리키게 됩니다.

31 : np.max 함수를 호출하여 window가 가리키는 행렬의 항목 중에 최대값을 갖는 항목을 max_pooled 행렬의 (row, col) 항목에 할당합니다.

33 : images 리스트에 max_pooled를 추가합니다.

34 : labels 리스트에 'max_pooled' 문자열을 추가합니다.
36 : 그림의 가로 크기를 15인치로 변경합니다.
38 : 그림의 열의 개수를 3으로 변경합니다.

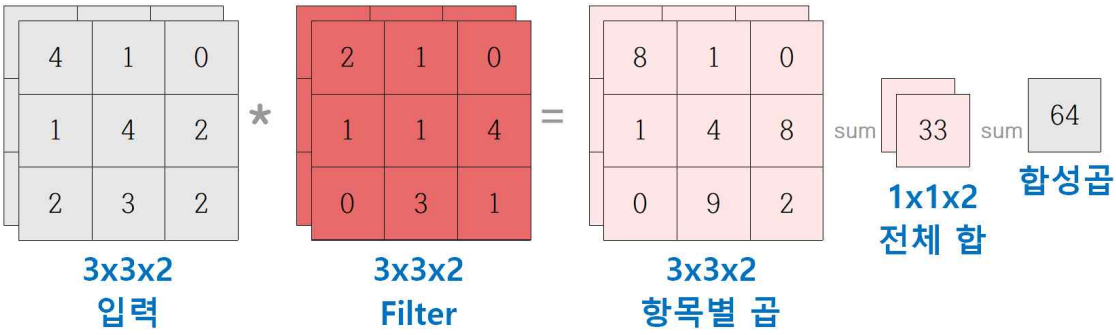
2. ▶ 버튼을 눌러 프로그램을 실행시킵니다. 다음은 실행 결과 화면입니다.



맨 오른쪽의 이미지는 중간 이미지에 대해 max pooling을 거쳐 단순화한 이미지입니다.

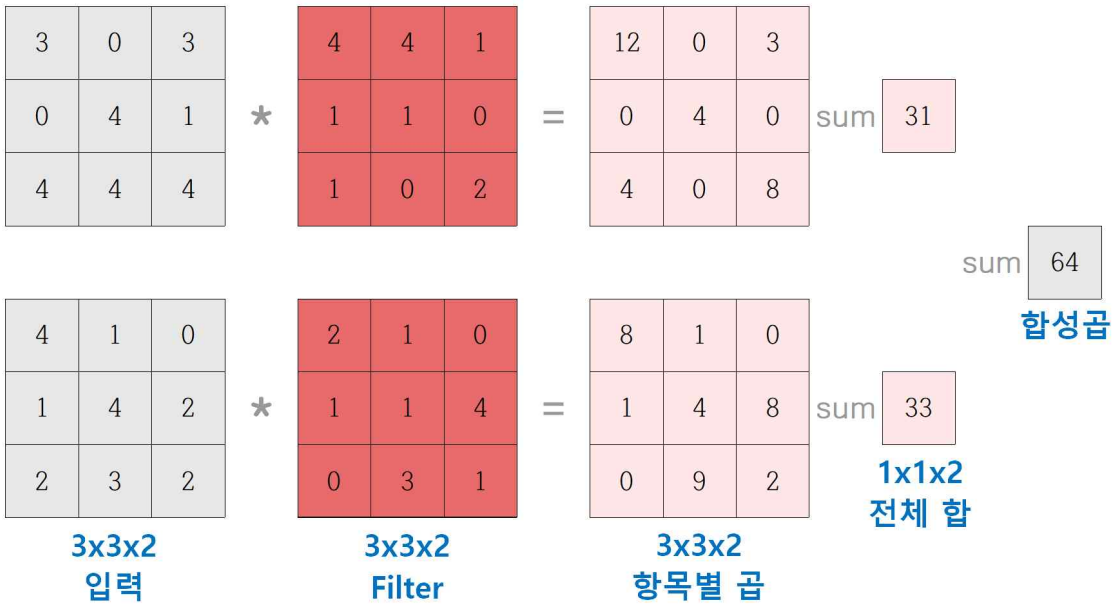
08 3x3x2 입력

여기서는 3x3x2 입력 이미지에 대해 합성 곱을 수행하는 과정을 살펴봅니다. 다음은 3x3x2 이미지에 대해 합성 곱을 수행하는 과정을 나타냅니다.



3x3x2 입력 이미지는 3x3x1 입력 이미지 2 장이 겹쳐진 상태로 입력되는 하나의 입체 이미지입니다. 겹쳐진 정도를 depth(깊이) 또는 channel이라고 합니다. 각각의 이미지는 RGB와 같이 색깔의 특징을 가진 이미지일 수도 있고, 다른 여러 가지 필터를 통해 추출된 특징을 가진 이미지일 수도 있습니다. 입력 이미지에 합성 곱을 위해 적용되는 필터의 두께(깊이)는 이미지의 두께와 같아야 합니다. 여기서 필터는 3x3x2로 이미지의 두께와 같습니다. 예를 들어, 이미지의 크기가 3x3x3이라면 필터의 크기는 3x3x3이 되며, 이미지의 크기가 3x3x4라면 필터의 크기도 3x3x4가 되어야 합니다. 각 층의 이미지와 필터는 각각의 대응되는 항목이 개별적으로 곱해져 항목별 곱을 구성한 후에 9개의 항목이 모두 더해져 중간 결과 값을 얻은 후에 층별로 더해진 값을 모두 더해 최종 결과 값 하나를 얻게 됩니다. 이 과정은 3, 4, 5 층

등 임의의 개수의 층에도 적용됩니다. 다음은 3x3x2 입력 이미지에 대해 구체적으로 수행되는 합성 곱 과정을 나타냅니다.



지금까지의 과정을 예제를 통해 정리해 봅니다.

1. 다음과 같이 예제를 작성합니다.

518_1.py

```
01 import numpy as np
02
03 np.random.seed(1)
04
05 image = np.random.randint(5, size=(3,3,2))
06 print('image_0 =\n', image[:,:,0])
07 print('image_1 =\n', image[:,:,1])
08
09 filter = np.random.randint(5, size=(3,3,2))
10 print('filter_0 =\n', filter[:,:,0])
11 print('filter_1 =\n', filter[:,:,1])
12
13 image_x_filter = image * filter
14 print('image_x_filter_0 =\n', image_x_filter[:,:,0])
15 print('image_x_filter_1 =\n', image_x_filter[:,:,1])
16
17 convolution = np.sum(image_x_filter)
18 print('convolution =\n', convolution)
```

05 : 0이상 5미만의 정수를 가진 3x3x2 행렬을 생성하여 image 변수에 할당합니다. 이 경우에는 2의 depth(깊이)를 갖는 3x3 행렬로 해석합니다.
06 : print 함수를 호출하여 image[:,:,0] 행렬 값을 출력합니다.
07 : print 함수를 호출하여 image[:,:,1] 행렬 값을 출력합니다.
09 : 0이상 5미만의 정수를 가진 3x3x2 행렬을 생성하여 filter 변수에 할당합니다. 이 경우에는 2의 depth(깊이)를 갖는 3x3 행렬로 해석합니다. 필터의 깊이 2는 입력 이미지의 깊이 2와 같아야 합니다.
10 : print 함수를 호출하여 filter[:,:,0] 행렬 값을 출력합니다.
11 : print 함수를 호출하여 filter[:,:,1] 행렬 값을 출력합니다.
13 : image와 filter를 곱한 후, 결과 행렬을 image_x_filter에 할당합니다. 파이썬에서 같은 크기를 갖는 두 개의 NumPy 행렬을 곱하면 같은 위치에 있는 항목끼리 곱셈이 수행됩니다. 3차원 행렬에 대해서도 마찬가지입니다.
14 : print 함수를 호출하여 image_x_filter[:,:,0] 행렬 값을 출력합니다.
15 : print 함수를 호출하여 image_x_filter[:,:,1] 행렬 값을 출력합니다.
17 : np.sum 함수를 호출하여 image_x_filter 행렬의 모든 항목 값을 더하여 convolution 변수에 할당합니다.
18 : print 함수를 호출하여 convolution 변수 값을 출력해 봅니다.

2. ▶ 버튼을 눌러 프로그램을 실행시킵니다. 다음은 실행 결과 화면입니다.

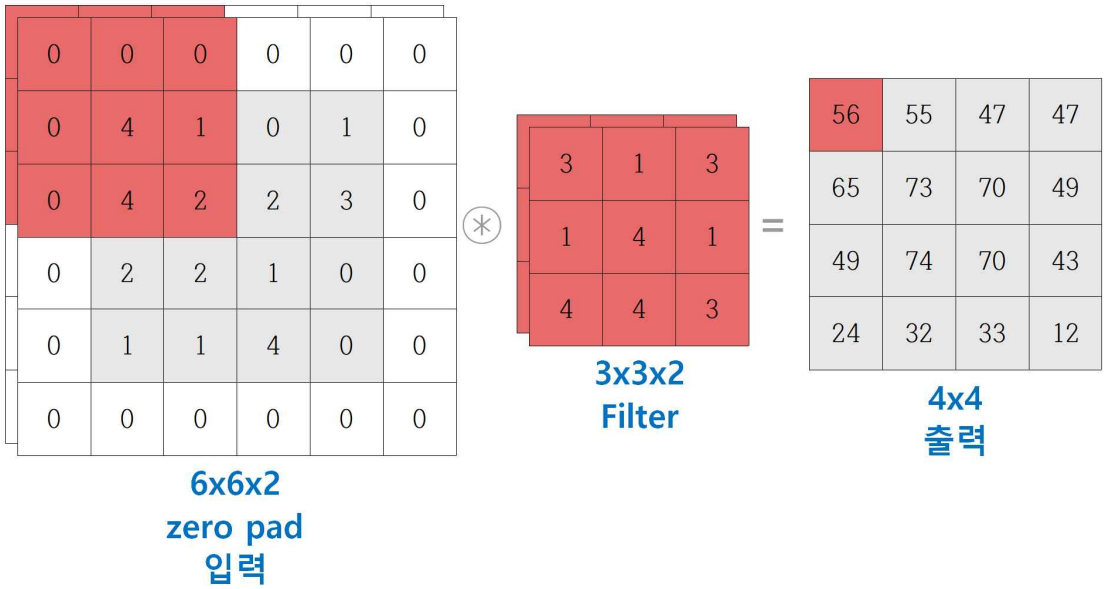


출력 결과를 앞의 그림과 비교하면서 살펴봅니다.

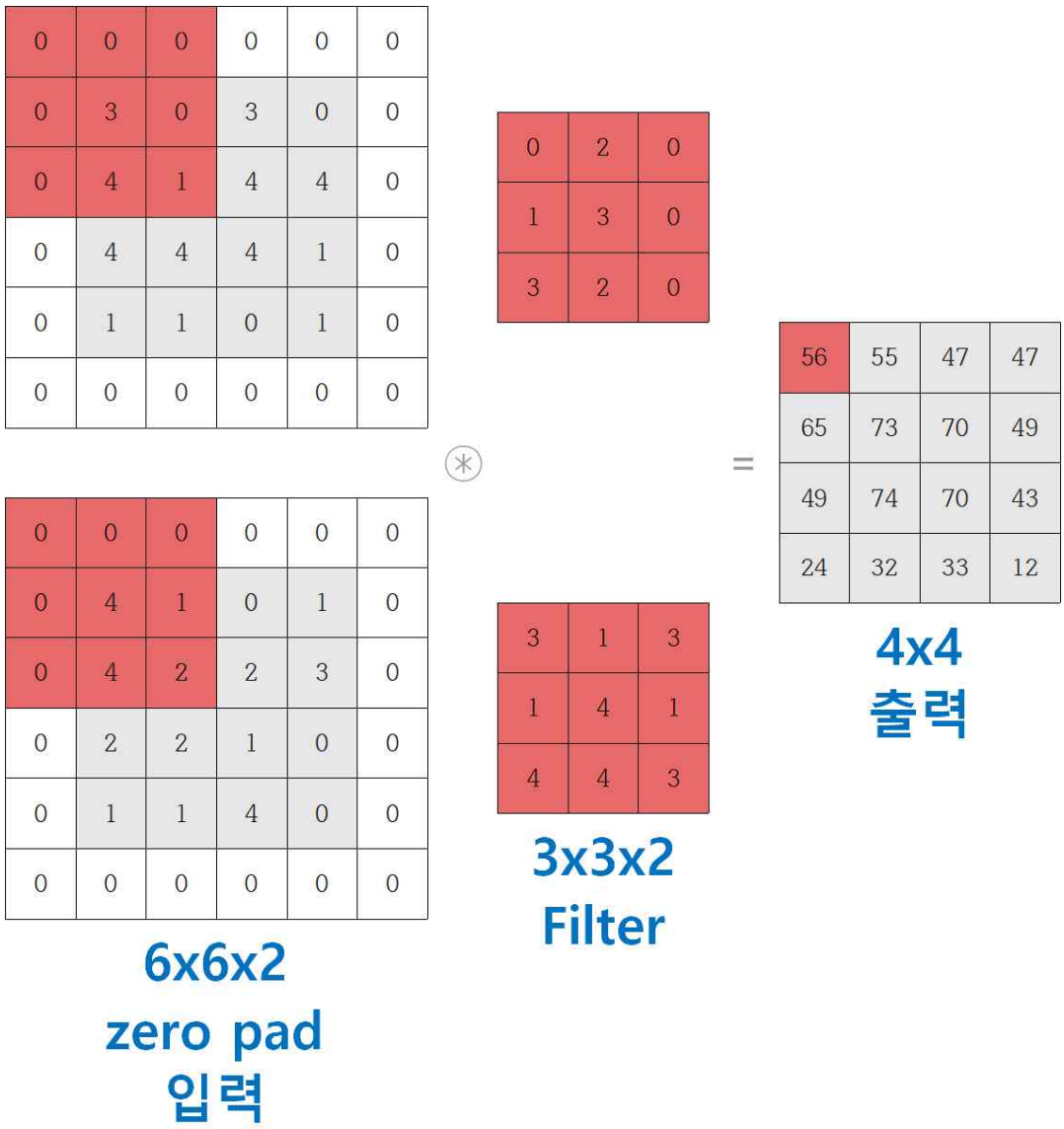
*** 독자의 이해를 돕기 위해 결과 화면을 편집하였습니다.

09 6x6x2 입력

다음은 4x4x2 입력 이미지를 상하 좌우로 1칸씩 zero padding을 하여 6x6x2 이미지로 변경한 후, 합성 곱을 수행한 결과를 보여줍니다.



6x6x2 입력 이미지는 3x3x2 필터와 합성 곱이 수행된 후, 4x4x1 출력 이미지를 생성합니다. 입력 이미지의 두께와 상관없이 같은 두께의 필터와 합성 곱이 수행된 후에는 하나의 결과 이미지를 생성하게 됩니다. 여기서 출력 이미지의 깊이는 생략되었습니다. 다음은 6x6x2 입력 이미지에 대해 구체적으로 수행되는 합성 곱 과정을 나타냅니다.



2층의 깊이로 구성된 6x6 입력 이미지 1개는 2층의 깊이로 구성된 3x3 필터 1개와 층별로 합성 곱이 수행된 후, 1층 깊이의 4x4 출력 이미지 1개를 생성해 내게 됩니다.

지금까지의 과정을 예제를 통해 정리해 봅니다.

1. 다음과 같이 예제를 작성합니다.

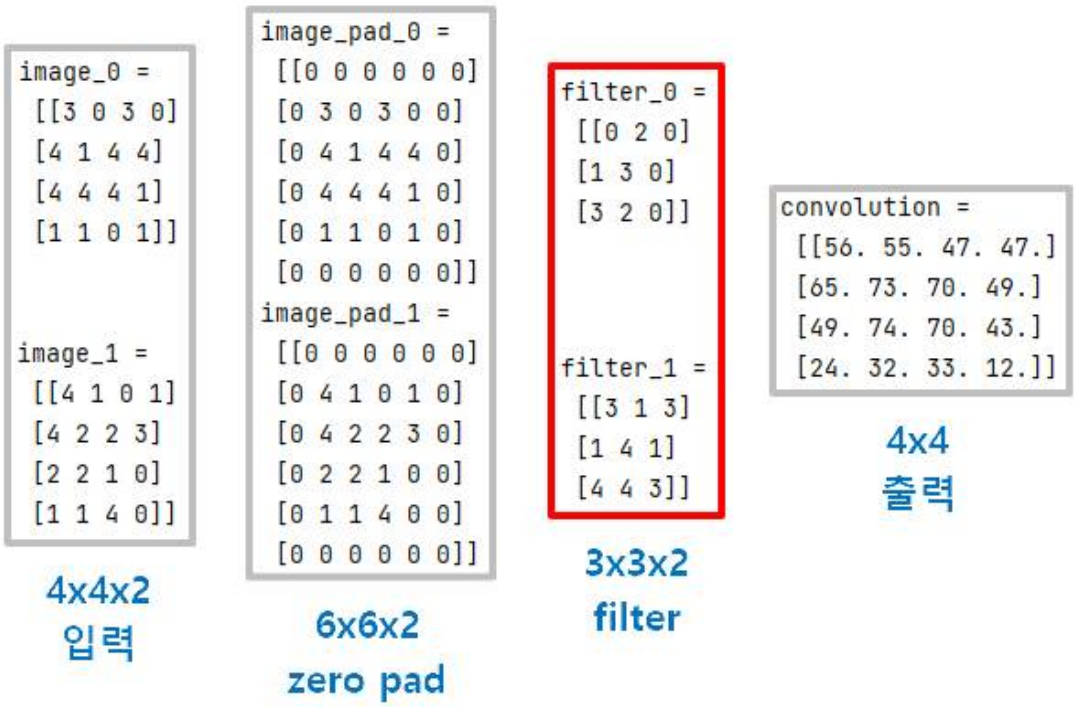
519_1.py

```
01 import numpy as np
02
03 np.random.seed(1)
04
05 image = np.random.randint(5, size=(4,4,2))
```

```
06 print('image_0 =\n', image[:,:,0])
07 print('image_1 =\n', image[:,:,1])
08
09 filter = np.random.randint(5, size=(3,3,2))
10 print('filter_0 =\n', filter[:,:,0])
11 print('filter_1 =\n', filter[:,:,1])
12
13 image_pad = np.pad(image,((1,1), (1,1), (0,0)))
14 print('image_pad_0 =\n', image_pad[:,:,0])
15 print('image_pad_1 =\n', image_pad[:,:,1])
16
17 convolution = np.zeros((4,4))
18
19 for row in range(4):
20       for col in range(4):
21             window = image_pad[row:row+3, col:col+3]
22             convolution[row, col] = np.sum(window*filter)
23
24 print('convolution =\n', convolution)
```

05 : 0이상 5미만의 정수를 가진 4x4x2 행렬을 생성하여 image 변수에 할당합니다. 이 경우에는 2의 depth(깊이)를 갖는 4x4 행렬로 해석합니다.
06 : print 함수를 호출하여 image[:,:,0] 행렬 값을 출력합니다.
07 : print 함수를 호출하여 image[:,:,1] 행렬 값을 출력합니다.
13 : np.pad 함수를 호출하여 image 행렬과 같은 모양의 행렬을 내부적으로 생성한 후, 행을 상하로 한 칸씩, 열을 좌우로 한 칸씩 늘려 0으로 채운 후, image_pad 변수에 할당합니다. 깊이 부분은 늘리지 않습니다.
14 : print 함수를 호출하여 image_pad[:,:,0] 행렬 값을 출력합니다.
15 : print 함수를 호출하여 image_pad[:,:,1] 행렬 값을 출력합니다.
17 : np.zeros 함수를 호출하여 0값으로 채워진 4x4 행렬을 생성하여 convolution 변수에 할당합니다.
19 : row값을 0에서 4 미만까지 바꾸어가며 20~22줄을 4회 수행합니다.
20 : col값을 0에서 4 미만까지 바꾸어가며 21~22줄을 4회 수행합니다.
21 : image_pad 행렬의 row 이상 (row+3) 미만까지, col 이상 (col+3) 미만까지의 행렬을 window 변수가 가리키게 합니다. 예를 들어, (row=0, col=0)일 경우 window 변수는 image_pad[0:3, 0:3] 행렬을 가리키게 됩니다.
22 : np.sum 함수를 호출하여 window*filter 행렬의 모든 항목의 값을 더해서 convolution 행렬의 (row, col) 항목에 할당합니다.
24 : print 함수를 호출하여 convolution 행렬 값을 출력합니다.

2.  버튼을 눌러 프로그램을 실행시킵니다. 다음은 실행 결과 화면입니다.

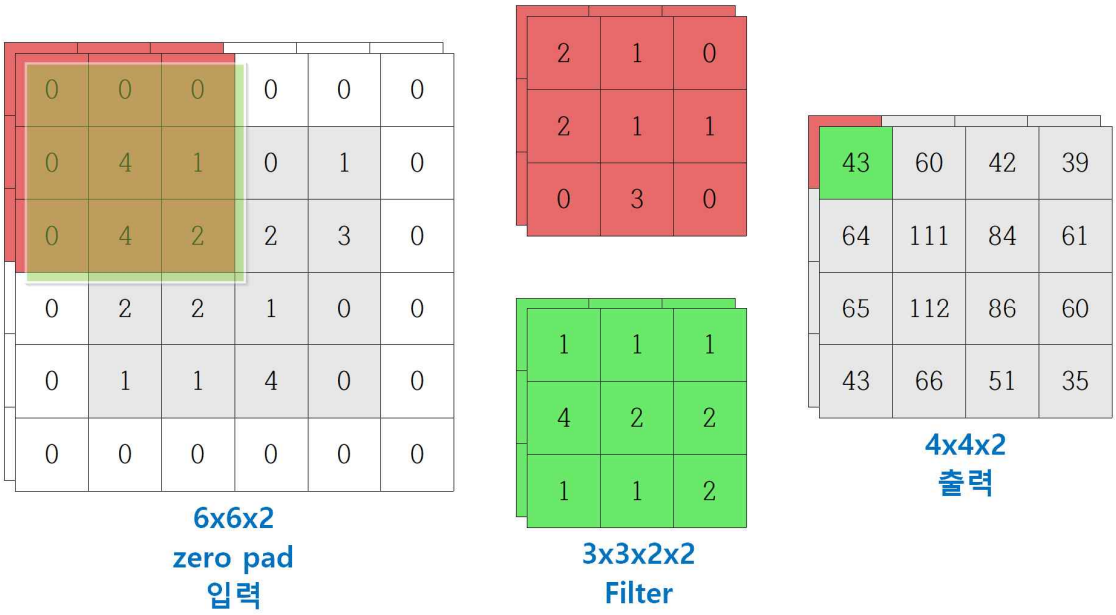


출력 결과를 앞의 그림과 비교하면서 살펴봅니다.

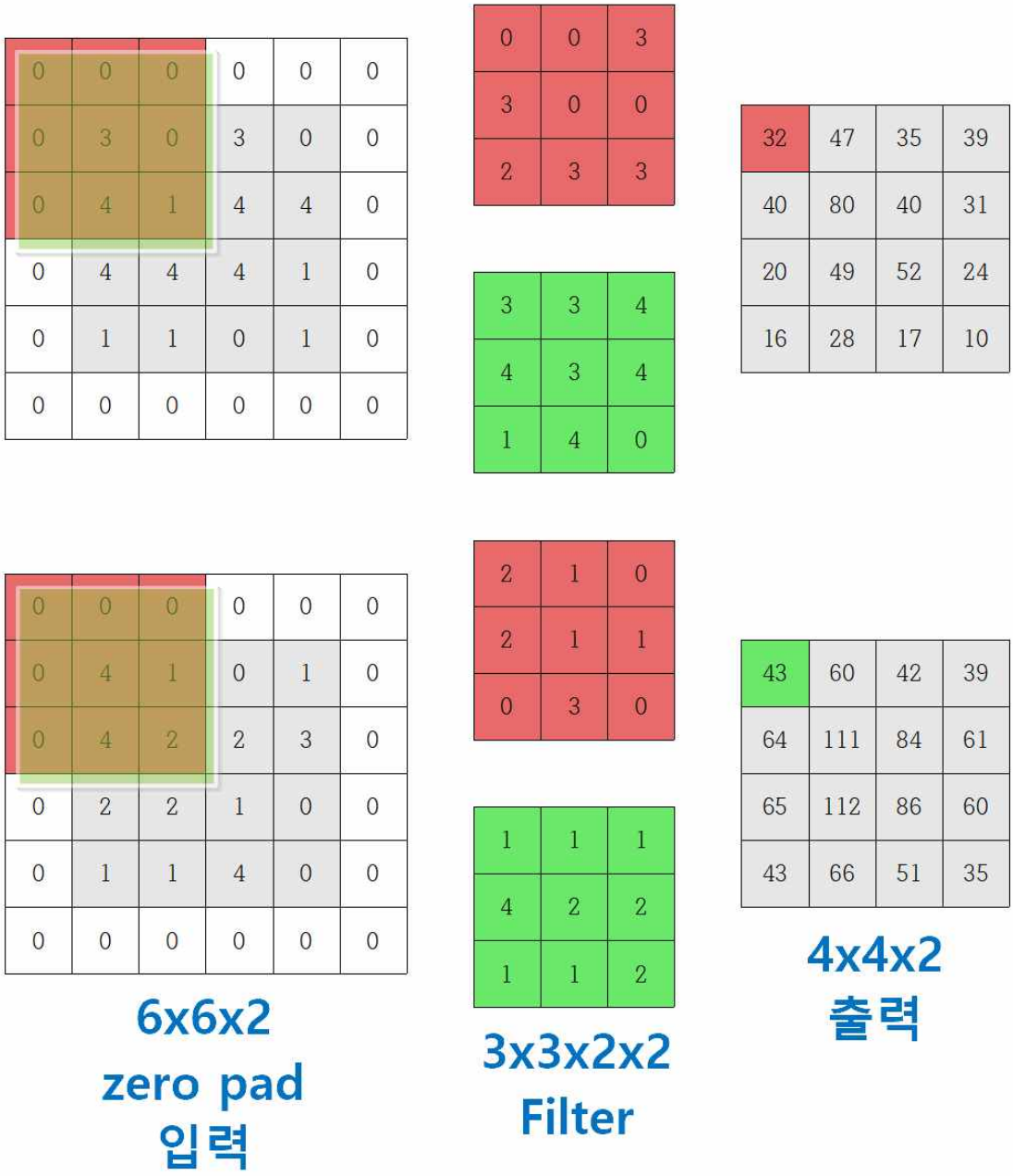
*** 독자의 이해를 돕기 위해 결과 화면을 편집하였습니다.

10 6x6x2 입력 필터 늘리기

다음은 6x6x2 입력 이미지에 대해 2개의 필터를 이용하여 2개의 출력 이미지를 얻는 과정을 보여줍니다. 필터는 이미지의 특징을 추출하는 역할을 하므로 여기서는 입력 이미지에서 2가지 특징을 추출하게 됩니다. 일반적으로 필터의 개수만큼 이미지의 특징이 추출됩니다. 필터의 두께는 입력 이미지의 두께와 같아야 합니다.



6x6x2 입력 이미지는 3x3x2 필터 2개와 합성 곱이 수행된 후, 4x4x2 출력 이미지를 생성합니다. 입력 이미지의 두께와 상관없이 같은 두께의 필터 하나와 합성 곱이 수행된 후에는 하나의 결과 이미지를 생성하게 됩니다. 여기서는 2개의 필터가 사용되었으므로 2개의 결과 이미지를 생성하며, 결과 이미지들은 겹쳐져 이미지의 개수만큼의 깊이를 갖는 하나의 이미지를 구성합니다. 다음은 6x6x2 입력 이미지에 대해 3x3x2 필터 2개를 적용하여 합성 곱을 수행하는 과정을 구체적으로 보여줍니다.



2층의 깊이로 구성된 6x6 입력 이미지 1개는 2층의 깊이로 구성된 3x3 필터 2개와 층별로 합성 곱이 수행된 후, 2층 깊이의 4x4 출력 이미지 1개를 생성해 내게 됩니다.

지금까지의 과정을 예제를 통해 정리해 봅니다.

1. 다음과 같이 예제를 작성합니다.

5110_1.py

```
01 import numpy as np
```

```
02
03 np.random.seed(1)
04
05 image = np.random.randint(5, size=(4,4,2))
06 print('image_0 =\n', image[:,:,0])
07 print('image_1 =\n', image[:,:,1])
08
09 filter = np.random.randint(5, size=(3,3,2,2))
10 print('filter_00 =\n', filter[:,:,0,0])
11 print('filter_10 =\n', filter[:,:,1,0])
12 print('filter_01 =\n', filter[:,:,0,1])
13 print('filter_11 =\n', filter[:,:,1,1])
14
15 image_pad = np.pad(image,((1,1), (1,1), (0,0)))
16 print('image_pad_0 =\n', image_pad[:,:,0])
17 print('image_pad_1 =\n', image_pad[:,:,1])
18
19 convolution = np.zeros((4,4,2))
20
21 for fn in range(2):
22     for row in range(4):
23         for col in range(4):
24             window = image_pad[row:row+3, col:col+3]
25             convolution[row, col, fn] = np.sum(window*filter[:,:,:,fn])
26
27 print('convolution_0 =\n', convolution[:,:,0])
28 print('convolution_1 =\n', convolution[:,:,1])
```

09 : 0이상 5미만의 정수를 가진 3x3x2x2 행렬을 생성하여 filter 변수에 할당합니다. 이렇게 하면 2의 깊이를 갖는 3x3 크기의 필터 2개가 생성됩니다. 세 번째에 오는 2는 필터의 깊이로, 마지막에 오는 2는 필터의 개수로 해석합니다.

10 : print 함수를 호출하여 0번 filter의 0번 깊이를 출력해 봅니다.

11 : print 함수를 호출하여 0번 filter의 1번 깊이를 출력해 봅니다.

12 : print 함수를 호출하여 1번 filter의 0번 깊이를 출력해 봅니다.

13 : print 함수를 호출하여 1번 filter의 1번 깊이를 출력해 봅니다.

19 : np.zeros 함수를 호출하여 0값으로 채워진 4x4x2 행렬을 생성하여 convolution 변수에 할당합니다. 이 경우에는 2의 depth(깊이)를 갖는 4x4 행렬로 해석합니다. convolution 행렬의 깊이 2는 09줄에서 정의된 filter 행렬의 개수 2와 같아야 합니다.

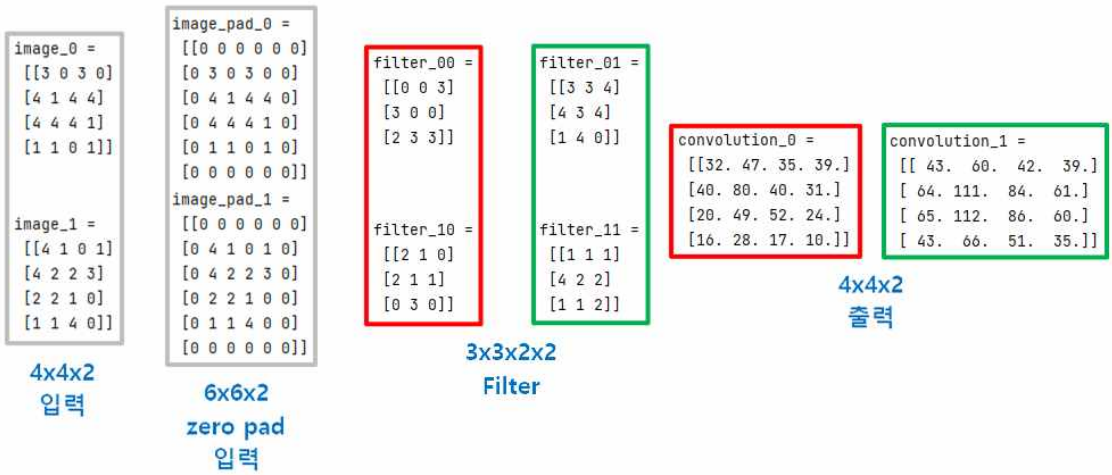
21 : fn값을 0에서 2 미만까지 바꾸어가며 22~25줄을 2회 수행합니다. fn은 filter number를 의미합니다.

22 : row값을 0에서 4 미만까지 바꾸어가며 23~25줄을 4회 수행합니다.

23 : col값을 0에서 4 미만까지 바꾸어가며 24~25줄을 4회 수행합니다.

24 : image_pad 행렬의 row 이상 (row+3) 미만까지, col 이상 (col+3) 미만까지의 행렬을 window 변수가 가리키게 합니다. 예를 들어, (row=0, col=0)일 경우 window 변수는 image_pad[0:3, 0:3] 행렬을 가리키게 됩니다.
25 : np.sum 함수를 호출하여 window*filter[:,:,:,fn] 행렬의 모든 항목의 값을 더해서 convolution 행렬의 (row, col, fn) 항목에 할당합니다.
27 : print 함수를 호출하여 convolution[:,:,0] 행렬 값을 출력합니다.
28 : print 함수를 호출하여 convolution[:,:,1] 행렬 값을 출력합니다.

2. ▶ 버튼을 눌러 프로그램을 실행시킵니다. 다음은 실행 결과 화면입니다.



출력 결과를 앞의 그림과 비교하면서 살펴봅니다.

*** 독자의 이해를 돕기 위해 결과 화면을 편집하였습니다.

11 6x6x3 입력

여기서는 6x6x3 입력 이미지에 대해 2개의 필터를 이용하여 2개의 출력 이미지를 얻는 예제를 수행하여 봅니다.

1. 다음과 같이 예제를 작성합니다.

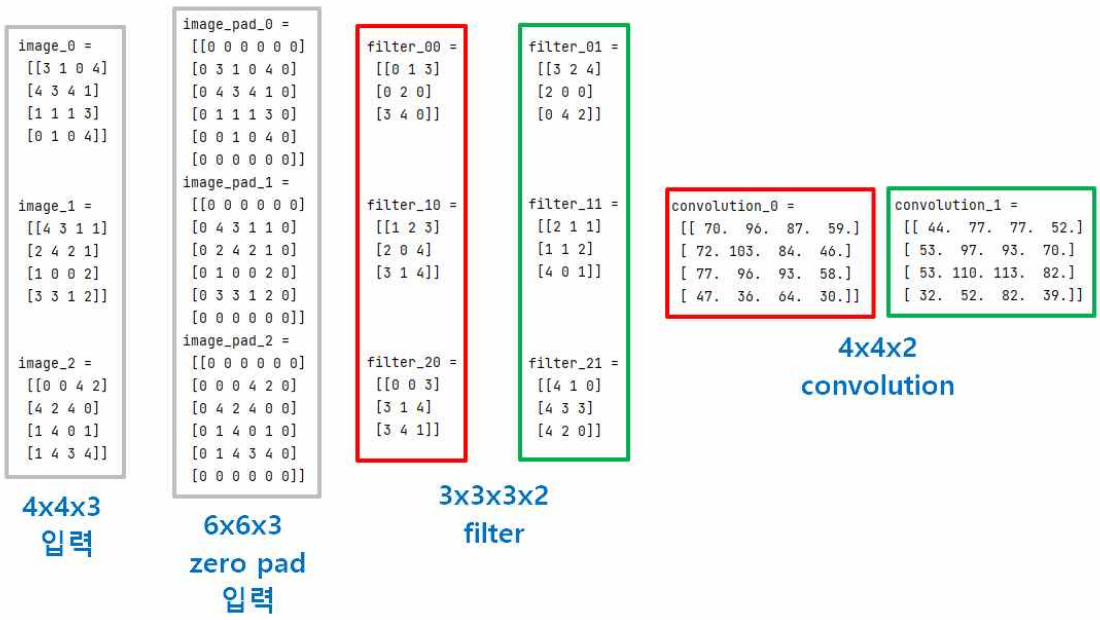
5111_1.py

```
01 import numpy as np
02
03 np.random.seed(1)
04
05 image = np.random.randint(5, size=(4,4,3))
06 print('image_0 =\n', image[:,:,0])
07 print('image_1 =\n', image[:,:,1])
```

```
08 print('image_2 =\n', image[:,:,2])
09
10 filter = np.random.randint(5, size=(3,3,3,2))
11 print('filter_00 =\n', filter[:,:,0,0])
12 print('filter_01 =\n', filter[:,:,1,0])
13 print('filter_02 =\n', filter[:,:,2,0])
14 print('filter_10 =\n', filter[:,:,0,1])
15 print('filter_11 =\n', filter[:,:,1,1])
16 print('filter_12 =\n', filter[:,:,2,1])
17
18 image_pad = np.pad(image,((1,1), (1,1), (0,0)))
19 print('image_pad_0 =\n', image_pad[:,:,0])
20 print('image_pad_1 =\n', image_pad[:,:,1])
21 print('image_pad_2 =\n', image_pad[:,:,2])
22
23 convolution = np.zeros((4,4,2))
24
25 for fn in range(2):
26     for row in range(4):
27         for col in range(4):
28             window = image_pad[row:row+3, col:col+3]
29             convolution[row, col, fn] = np.sum(window*filter[:,:,:,fn])
30
31 print('convolution_0 =\n', convolution[:,:,0])
32 print('convolution_1 =\n', convolution[:,:,1])
```

05 : 입력 이미지의 깊이를 3으로 바꿔 줍니다.
08 : 깊이 2의 이미지 출력 부분을 추가합니다.
10 : 필터의 깊이를 3으로 바꿔줍니다. 필터의 깊이는 입력 이미지의 깊이와 같아야 합니다.
13 : 0번 filter의 2번 깊이 출력 부분을 추가합니다.
16 : 1번 filter의 2번 깊이 출력 부분을 추가합니다.
21 : image_pad의 2번 깊이 출력 부분을 추가합니다.
23 : np.zeros 함수를 호출하여 0값으로 채워진 4x4x2 행렬을 생성하여 convolution 변수에 할당합니다. convolution 행렬의 깊이 2는 10줄에서 정의된 filter 행렬의 개수 2와 같아야 합니다.

2.  버튼을 눌러 프로그램을 실행시킵니다. 다음은 실행 결과 화면입니다.

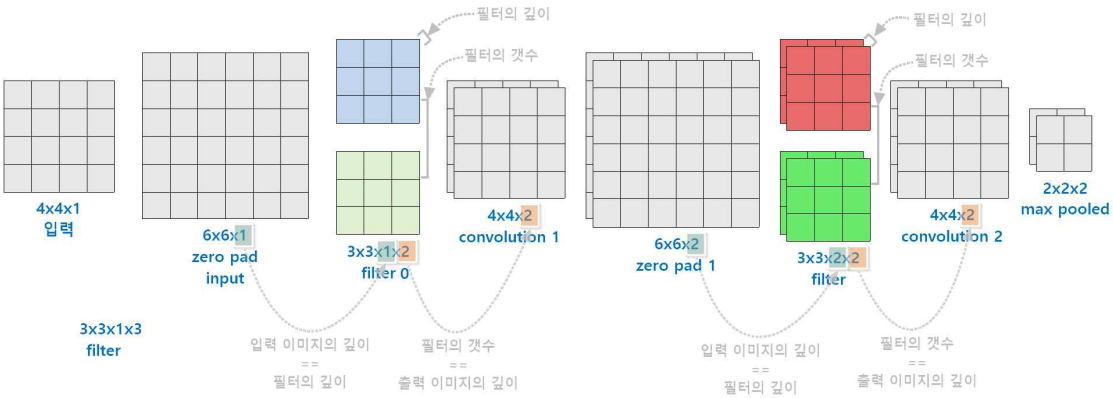


입력의 깊이는 필터의 깊이를 결정하여 필터의 개수는 출력의 깊이를 결정합니다.

*** 독자의 이해를 돕기 위해 결과 화면을 편집하였습니다.

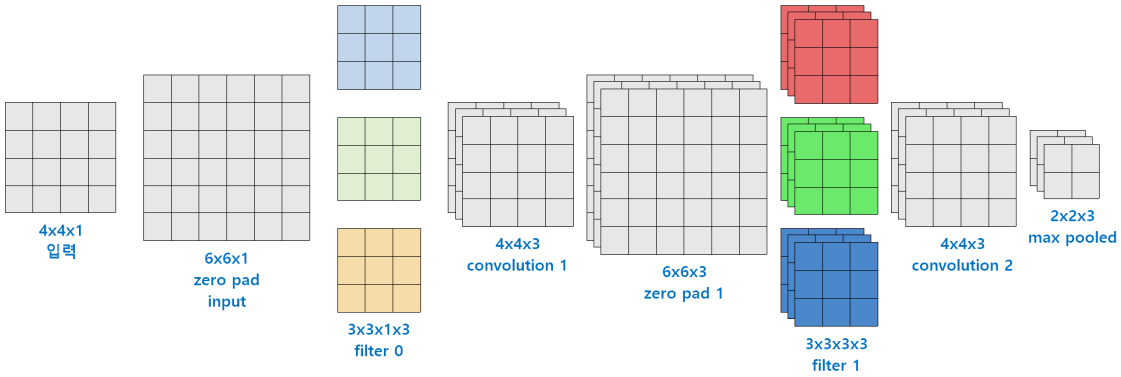
12 필터의 깊이와 개수

다음 그림은 입력 이미지의 깊이와 필터의 깊이, 필터의 개수와 출력 이미지의 깊이의 관계를 보여줍니다.



입력의 깊이는 필터의 깊이를 결정하여 필터의 개수는 출력의 깊이를 결정합니다.

다음 그림은 필터의 개수가 늘어났을 때, 출력의 깊이가 늘어나는 상황을 보여줍니다.



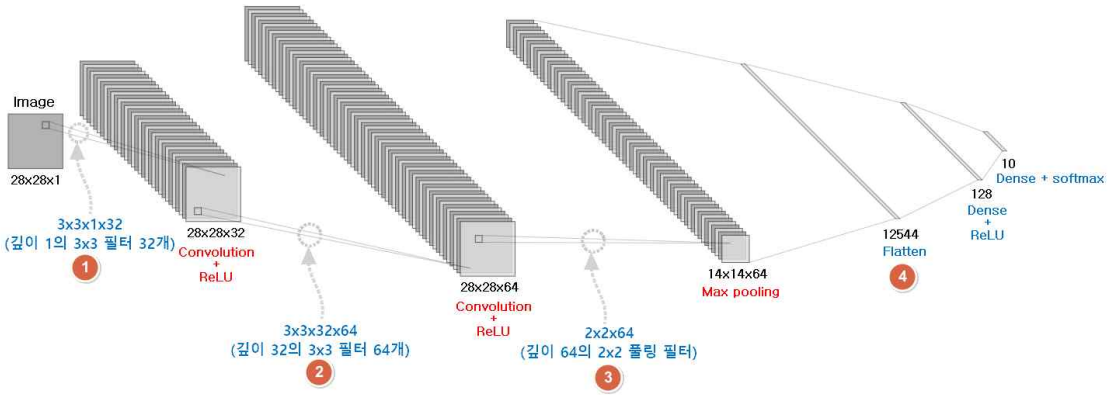
이상 CNN의 순전파 과정을 살펴보았습니다. 뒤에서 우리는 CNN 역전파 과정도 살펴봅니다.

02 CNN 활용 맛보기

앞에서 우리는 CNN의 주요 동작을 살펴보고 구현해 보았습니다. 여기서는 CNN 인공 신경망을 학습시켜 숫자와 그림을 인식해 보도록 합니다. CNN 인공 신경망은 앞에서 살펴본 완전 연결 인공 신경망에 비해 이미지 인식률이 훨씬 높습니다. 그러나 학습 시간은 훨씬 더 길다는 단점이 있습니다.

01 Conv2D-Conv2D-MaxPooling2D

다음은 합성 곱 2단계, 모으기 1단계, 완전 연결 층으로 구성된 인공 신경망입니다.



이 신경망의 입력 층은 28x28x1의 이미지입니다. 입력 층의 깊이가 1이므로 사용할 필터의 깊이도 1이 되어야 합니다. 1차 합성 곱 출력 층은 28x28x32로 깊이가 32입니다. 출력 층의 깊이가 32일 경우 사용할 필터의 개수는 32개가 되어야 합니다. ❶ 그래서 필터는 3x3x1x32의 4차원 행렬이 됩니다. 1차 합성 곱은 ReLU 함수를 거쳐 2차 합성 곱의 입력이 됩니다. 2차 합성 곱 입력 층의 깊이는 32이므로 사용할 필터의 깊이도 32가 되어야 합니다. 2차 합성 곱 출력 층은 28x28x64로 깊이가 64입니다. 출력 층의 깊이가 64이므로 사용할 필터의 개수는 64개가 되어야 합니다. ❷ 그래서 필터는 3x3x32x64의 4차원 행렬이 됩니다. 2차 합성 곱은 ReLU 함수를 거쳐 모으기의 입력이 됩니다. ❸ 모으기에 사용되는 필터는 2x2x64를 사용하여 모으기 출력 이미지는 14x14x64가 됩니다. ❹ 모으기 출력 이미지는 Flatten 과정을 거쳐 14x14x64 = 12544개의 1차원 이미지로 재구성됩니다.

이 신경망을 예제를 통해 살펴봅니다.

1. 다음과 같이 예제를 작성합니다.

521_1.py

```
01 import torch
02 import torch.nn as nn
03 from torch.utils.data import DataLoader
04 from torchvision import datasets, transforms
```

```
05
06 device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
07
08 # ToTensor(): uint8 [0,255] -> float [0,1], shape (1, 28, 28)  (Keras의 /255.0 + reshape에 해당)
09 transform = transforms.ToTensor()
10 train_ds = datasets.FashionMNIST(root="./data", train=True, download=True, transform=transform)
11 test_ds = datasets.FashionMNIST(root="./data", train=False, download=True, transform=transform)
12
13 # Keras fit()의 기본 batch_size=32, shuffle=True와 동일하게
14 train_loader = DataLoader(train_ds, batch_size=32, shuffle=True)
15 test_loader = DataLoader(test_ds, batch_size=32, shuffle=False)
16
17 model = nn.Sequential(
18         nn.Conv2d(1, 32, kernel_size=3, padding=1),    # padding='same'
19         nn.ReLU(),
20         nn.Conv2d(32, 64, kernel_size=3, padding=1),
21         nn.ReLU(),
22         nn.MaxPool2d(2),                               # 28x28 -> 14x14
23         nn.Flatten(),                                  # 64*14*14 = 12544
24         nn.Linear(64 * 14 * 14, 128),
25         nn.ReLU(),
26         nn.Linear(128, 10),                            # softmax 없음: CrossEntropyLoss가 내부에서 처리
27 ).to(device)
28
29 print(model)
30 print("Total params:", sum(p.numel() for p in model.parameters()))
31
32 criterion = nn.CrossEntropyLoss()                     # = softmax + sparse_categorical_crossentropy
33 optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)  # Keras Adam 기본값과 동일
34
35
36 def train_one_epoch():
37         model.train()
38         total_loss, correct, n = 0.0, 0, 0
39         for x, y in train_loader:
40                 x, y = x.to(device), y.to(device)
41                 optimizer.zero_grad()
42                 logits = model(x)
43                 loss = criterion(logits, y)
44                 loss.backward()
45                 optimizer.step()
46                 total_loss += loss.item() * x.size(0)
47                 correct += (logits.argmax(1) == y).sum().item()
48                 n += x.size(0)
49         return total_loss / n, correct / n
50
51
52 @torch.no_grad()
53 def evaluate(loader):
54         model.eval()
55         total_loss, correct, n = 0.0, 0, 0
```

```
56          for x, y in loader:
57                      x, y = x.to(device), y.to(device)
58                      logits = model(x)
59                      total_loss += criterion(logits, y).item() * x.size(0)
60                      correct += (logits.argmax(1) == y).sum().item()
61                      n += x.size(0)
62          return total_loss / n, correct / n
63
64
65 for epoch in range(5):
66          loss, acc = train_one_epoch()
67          print(f"Epoch {epoch + 1}/5 - loss: {loss:.4f} - accuracy: {acc:.4f}")
68
69 test_loss, test_acc = evaluate(test_loader)
70 print(f"Test - loss: {test_loss:.4f} - accuracy: {test_acc:.4f}")
```

라이브러리 임포트 (01-04)

01: import torch: PyTorch의 핵심 패키지입니다. 텐서 연산, 자동 미분, GPU 연산을 제공합니다.
02: import torch.nn as nn: 신경망 계층(Conv2d, Linear 등)과 손실 함수를 모아 둔 모듈을 nn이라는 이름으로 가져옵니다.
03: from torch.utils.data import DataLoader: 데이터셋을 미니배치 단위로 묶고 섞어서 공급하는 반복자 클래스입니다.
04: from torchvision import datasets, transforms: torchvision의 데이터셋 모음(datasets)과 이미지 전처리 도구(transforms)를 가져옵니다.

연산 장치 선택 (06)

06: CUDA GPU를 쓸 수 있으면 "cuda", 아니면 "cpu"를 연산 장치로 정합니다. 이후 모델과 데이터를 모두 이 장치로 옮겨야 연산이 가능합니다.

데이터 준비 (08-15)

08: 주석입니다. 다음 줄의 ToTensor()가 하는 일을 설명합니다.
09: transforms.ToTensor(): 원본 이미지를 텐서로 바꾸는 변환입니다. 세 가지가 함께 일어납니다.

- 값 범위: uint8 0-255 → float32 0.0-1.0
- 차원 순서: (H, W) → (C, H, W) = (1, 28, 28)
- Keras 코드의 /255.0과 reshape를 한 번에 처리합니다.

10: 학습용 Fashion-MNIST 6만 장을 불러옵니다. 인자의 의미는 다음과 같습니다.

- root="./data": 저장 위치
- train=True: 학습 세트 선택

- download=True: 없으면 자동 다운로드
- transform: 샘플을 꺼낼 때마다 적용할 전처리

11: 같은 방식으로 테스트용 1만 장을 불러옵니다(train=False).
13: 주석입니다. Keras fit()의 기본값에 맞췄다는 설명입니다.
14: 학습 데이터를 32장씩 묶는 DataLoader를 만듭니다. shuffle=True이므로 epoch마다 순서를 섞어 학습이 특정 순서에 편향되지 않게 합니다.
15: 테스트용 DataLoader입니다. 평가는 순서가 결과에 영향을 주지 않으므로 섞지 않습니다.

모델 정의 (17-27)

17: nn.Sequential(: 나열된 계층을 순서대로 통과시키는 컨테이너입니다. Keras Sequential에 해당합니다.
18: nn.Conv2d(1, 32, kernel_size=3, padding=1): 첫 번째 합성곱 층입니다.
- 입력 1채널(흑백)에 3×3 필터 32개를 적용해 32채널 특징 맵을 만듭니다.
- padding=1이 가장자리를 0으로 한 칸 채우므로 출력 크기가 28×28로 유지됩니다. Keras의 padding='same'과 같습니다.
- 파라미터 수: 1×3×3×32 + 32 = 320

19: nn.ReLU(): 음수를 0으로 만드는 활성화 함수입니다. 비선형성을 더합니다. Keras는 activation='relu'로 층 안에 쓰지만, PyTorch는 별도 계층으로 둡니다.
20: nn.Conv2d(32, 64, ...): 두 번째 합성곱 층입니다.
- 32채널을 받아 64채널을 출력하며, 크기는 28×28로 유지됩니다.
- 파라미터 수: 32×3×3×64 + 64 = 18,496

21: nn.ReLU(): 두 번째 활성화 함수입니다.
22: nn.MaxPool2d(2): 2×2 영역마다 최댓값만 남깁니다. 크기가 28×28 → 14×14로 줄어 연산량이 감소하고, 작은 위치 변화에 둔감해집니다.
23: nn.Flatten(): (배치, 64, 14, 14) 텐서를 (배치, 12544)로 펼칩니다. 첫 번째 배치 차원은 유지됩니다.
24: nn.Linear(64 * 14 * 14, 128): 12544개 입력을 128개 뉴런에 전부 연결하는 완전연결층입니다.
- 파라미터 수: 12544×128 + 128 = 1,605,760
- 모델 전체 파라미터의 약 99%가 이 층에 있습니다.

25: nn.ReLU(): 완전연결층 뒤의 활성화 함수입니다.
26: nn.Linear(128, 10): 출력층입니다. 10개 클래스 각각의 점수(logit)를 출력합니다.
- 파라미터 수: 128×10 + 10 = 1,290
- Keras와 달리 softmax를 붙이지 않습니다. 손실 함수가 내부에서 처리하기 때문입니다.

27:).to(device): 완성된 모델의 모든 파라미터를 06행에서 고른 장치(GPU/CPU)로 옮깁니다.

모델 정보 출력 (29-30)

29: print(model): 계층 구성을 출력합니다. Keras model.summary()와 비슷하지만 출력 크

기나 파라미터 수는 보여 주지 않습니다.
30: 모든 파라미터 텐서의 원소 수(numel())를 더해 전체 파라미터 수를 출력합니다. 결과는 1,625,866개로, Keras summary와 같습니다.

손실 함수와 옵티마이저 (32-33)

32: nn.CrossEntropyLoss(): 다중 분류용 손실 함수입니다.
- logits에 log-softmax를 적용한 뒤, 정답 클래스의 음의 로그 확률을 계산합니다.
- 정답 y는 원-핫이 아니라 정수 레이블(0-9)을 그대로 받습니다. 그래서 Keras의 softmax + sparse_categorical_crossentropy 조합과 같습니다.

33: Adam 옵티마이저를 만듭니다. model.parameters()로 갱신할 파라미터를 넘기고, 학습률은 Keras 기본값과 같은 0.001로 둡니다.

학습 함수 (36-49)

36: 1 epoch(전체 학습 데이터 1회) 학습을 수행하는 함수를 정의합니다.
37: model.train(): 모델을 학습 모드로 전환합니다. Dropout과 BatchNorm이 학습용으로 동작하게 하는 스위치입니다. 지금 모델에는 해당 층이 없지만 습관적으로 넣는 것이 좋습니다.
38: 누적 손실, 맞힌 개수, 처리한 샘플 수를 0으로 초기화합니다.
39: DataLoader에서 이미지 배치 x (32,1,28,28)와 레이블 배치 y (32,)를 하나씩 꺼냅니다. 6만 / 32 = 1875회 반복합니다.
40: 배치를 모델과 같은 장치로 옮깁니다. 장치가 다르면 에러가 납니다.
41: optimizer.zero_grad(): 이전 배치에서 계산된 기울기를 0으로 지웁니다. PyTorch는 기울기를 누적하므로, 지우지 않으면 이전 배치의 기울기가 더해집니다.
42: logits = model(x): 순전파입니다. 배치를 모델에 통과시켜 (32, 10) 크기의 logit을 얻습니다.
43: logit과 정답으로 배치 평균 손실을 계산합니다.
44: loss.backward(): 역전파입니다. 자동 미분으로 모든 파라미터의 기울기를 계산해 각 파라미터의 .grad에 저장합니다.
45: optimizer.step(): 저장된 기울기를 이용해 Adam 규칙으로 파라미터를 한 번 갱신합니다.
46: loss.item()으로 텐서를 파이썬 숫자로 꺼냅니다. 이 값은 배치 평균이므로 배치 크기를 곱해 합계로 누적합니다. 마지막 배치는 32개보다 작을 수 있어(60000 / 32의 나머지) 이렇게 해야 정확합니다.
47: 클래스 차원(1)에서 가장 큰 logit의 인덱스(argmax)를 예측값으로 삼습니다. 정답과 같은 개수를 세어 누적합니다.
48: 처리한 샘플 수를 누적합니다.
49: epoch 전체의 평균 손실과 정확도를 반환합니다.

평가 함수 (52-62)

52: @torch.no_grad(): 이 함수 안에서는 기울기를 계산하지 않도록 하는 데코레이터입니다. 메모리와 연산이 절약되며, Keras evaluate()에서 자동으로 일어나는 일을 직접 지정하는 것입니다.
53: 주어진 DataLoader로 손실과 정확도를 계산하는 함수를 정의합니다.
54: model.eval(): 모델을 평가 모드로 전환합니다. 37행과 짝을 이룹니다.
55: 누적 변수를 초기화합니다.
56: 평가 데이터를 배치 단위로 꺼냅니다.
57: 배치를 장치로 옮깁니다.
58: 순전파만 수행합니다. 역전파와 갱신은 하지 않습니다.
59: 배치 손실 × 배치 크기를 누적합니다(46행과 같은 이유).
60: 맞힌 개수를 누적합니다.
61: 샘플 수를 누적합니다.
62: 평균 손실과 정확도를 반환합니다.

실행부 (65-70)

65: 5 epoch 동안 반복합니다. Keras의 epochs=5에 해당합니다.
66: 1 epoch를 학습하고 그 epoch의 평균 손실과 정확도를 받습니다.
67: Keras의 학습 로그와 비슷한 형식으로 결과를 출력합니다. :.4f는 소수점 넷째 자리까지 표시하라는 뜻입니다.
69: 학습이 끝난 모델을 테스트 세트로 평가합니다. Keras의 model.evaluate()에 해당합니다.
70: 테스트 손실과 정확도를 출력합니다.

2. ▶ 버튼을 눌러 프로그램을 실행시킵니다. 다음은 실행 결과 화면입니다.

```
Sequential(
  (0): Conv2d(1, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1)) ❶
  (1): ReLU()
  (2): Conv2d(32, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1)) ❷
  (3): ReLU()
  (4): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  (5): Flatten(start_dim=1, end_dim=-1) ❸
  (6): Linear(in_features=12544, out_features=128, bias=True) ❹
  (7): ReLU()
  (8): Linear(in_features=128, out_features=10, bias=True) ❺
)
Total params: 1625866 ❻
```

❶ 1차 합성 곱 단계에서 사용되는 가중치는 3x3x1x32의 크기를 갖습니다. 편향은 가중치의 개수만큼 필요하여 32개가 됩니다. 그래서 1차 합성 곱에 필요한 매개변수의 총 개수는 (3x3x1x32+32)=320이 됩니다. ❷ 2차 합성 곱 단계에서 사용되는 가중치는 3x3x32x64의 크기를 갖습니다. 편향은 가중치의 개수만큼 필요하여 64개가 됩니다. 그래서 2차 합성 곱에 필요한 매개변수의 총 개수는 (3x3x32x64+64)=18496이 됩니다. ❸ Flatten 층은 완전 연결 층

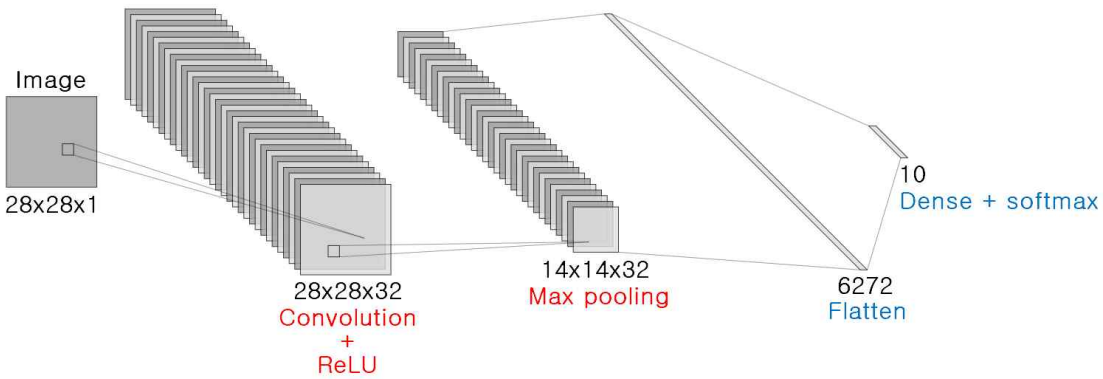
의 입력 층으로 14x14x64=12544개의 노드로 구성됩니다. ❹ 은닉 층의 노드는 128개로 Flatten 층과 은닉 층 연결에 필요한 가중치의 개수는 12544x128이 됩니다. 편향은 가중치의 개수만큼 필요하여 128개가 됩니다. 그래서 Flatten 층과 은닉 층 연결에 필요한 매개변수의 총 개수는 (12544x128+128)=1605760이 됩니다. ❺ 출력 층의 노드는 10개로 은닉 층과 출력 층 연결에 필요한 가중치의 개수는 128x10이 됩니다. 편향은 가중치의 개수만큼 필요하여 10개가 됩니다. 그래서 은닉 층과 출력 층 연결에 필요한 매개변수의 총 개수는 (128x10+10)=1290이 됩니다. ❻ 그래서 학습의 대상이 되는 매개변수의 총 개수는 1625866이 됩니다.

```
Epoch 1/5 - loss: 0.3681 - accuracy: 0.8660 (1)
Epoch 2/5 - loss: 0.2205 - accuracy: 0.9183
Epoch 3/5 - loss: 0.1685 - accuracy: 0.9376
Epoch 4/5 - loss: 0.1261 - accuracy: 0.9531
Epoch 5/5 - loss: 0.0915 - accuracy: 0.9668 (2)
Test - loss: 0.2500 - accuracy: 0.9255 (3)
```

패션 MNIST 데이터의 학습 과정의 정확도는 ❶ 86.60%에서 시작해서 ❷ 96.68%로 끝납니다. 완전 연결 인공 신경망에 비해 7% 정도 정확도가 높습니다. ❸ 학습이 끝난 후에, evalueate 함수로 시험 데이터를 예측한 결과는 92.55%로 학습 데이터의 예측 결과에 비해 정확도가 떨어진 상태입니다. 시험 데이터에 대한 예측 결과도 완전 연결 인공 신경망에 비해 6% 정도 정확도가 높습니다.

02 Conv2D-MaxPooling2D

다음은 합성 곱 1단계, 모으기 1단계, 완전 연결 층으로 구성된 인공 신경망입니다.



이전 인공 신경망에 비해 합성 곱 층은 1단계, 완전 연결 층의 은닉 층이 빠진 상태입니다.

이 신경망을 예제를 통해 살펴봅니다.

1. 다음과 같이 예제를 수정합니다.

522_1.py

```
17 model = nn.Sequential(
18         nn.Conv2d(1, 32, kernel_size=3, padding=1),    # padding='same'
19         nn.ReLU(),
20         nn.MaxPool2d(2),                               # 28x28 -> 14x14
21         nn.Flatten(),                                  # 32*14*14 = 6272
22         nn.Linear(32 * 14 * 14, 10),                   # softmax 없음: CrossEntropyLoss가 내부에서 처리
23 ).to(device)
```

20 : nn.MaxPoo2d를 이용하여 풀링 층을 생성합니다. 풀링 층에 사용할 풀링 필터의 크기는 2x2입니다. 이전 합성 곱 층의 깊이가 32이므로 풀링 층의 깊이도 32가 되어야 하며, 이미지의 크기는 반으로 줄어 14x14가 됩니다.

21 : nn.Flatten을 이용하여 이전 풀링 층을 완전 연결 층의 입력 층으로 모양을 변경합니다. 이 과정에서 14*14*32=6272개의 입력 노드로 변경됩니다.

2. ▶ 버튼을 눌러 프로그램을 실행시킵니다. 다음은 실행 결과 화면입니다.

```
Sequential(
  (0): Conv2d(1, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1)) ❶
  (1): ReLU()
  (2): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  (3): Flatten(start_dim=1, end_dim=-1) ❷
  (4): Linear(in_features=6272, out_features=10, bias=True) ❸
)
Total params: 63050 ❹
```

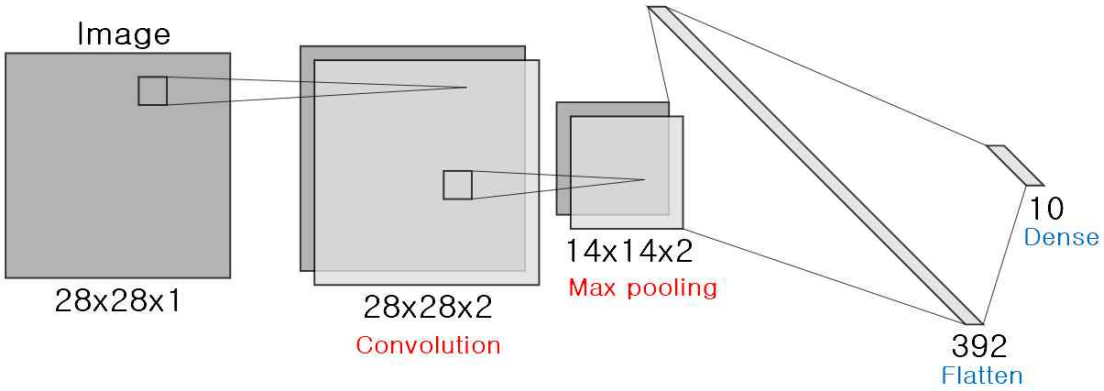
❶ 합성 곱 단계에서 사용되는 가중치는 3x3x1x32의 크기를 갖습니다. 편향은 가중치의 개수만큼 필요하여 32개가 됩니다. 그래서 합성 곱에 필요한 매개변수의 총 개수는 (3x3x1x32+32)=320이 됩니다. ❷ Flatten 층은 완전 연결 층의 입력 층으로 14x14x32=6272개의 노드로 구성됩니다. ❸ 출력 층의 노드는 10개로 Flatten 층과 출력 층 연결에 필요한 가중치의 개수는 6272x10이 됩니다. 편향은 가중치의 개수만큼 필요하여 10개가 됩니다. 그래서 Flatten 층과 출력 층 연결에 필요한 매개변수의 총 개수는 (6272x10+10)=62730이 됩니다. ❹ 그래서 학습의 대상이 되는 매개변수의 총 개수는 63050이 됩니다.

```
Epoch 1/5 - loss: 0.4450 - accuracy: 0.8429 ❶
Epoch 2/5 - loss: 0.3132 - accuracy: 0.8891
Epoch 3/5 - loss: 0.2767 - accuracy: 0.9014
Epoch 4/5 - loss: 0.2507 - accuracy: 0.9112
Epoch 5/5 - loss: 0.2308 - accuracy: 0.9167 ❷
Test - loss: 0.2704 - accuracy: 0.9045 ❸
```

패션 MNIST 데이터의 학습 과정의 정확도는 ❶ 84.29%에서 시작해서 ❷ 91.67%로 끝납니다. 완전 연결 인공 신경망에 비해 2% 정도 정확도가 높습니다. ❸ 학습이 끝난 후에, evalueate 함수로 시험 데이터를 예측한 결과는 90.45%로 학습 데이터의 예측 결과에 비해 정확도가 떨어진 상태입니다. 시험 데이터에 대한 예측 결과도 완전 연결 인공 신경망에 비해 2% 정도 정확도가 높습니다.

03 필터 개수 줄여보기

다음은 합성 곱 1단계, 모으기 1단계, 완전 연결 층으로 구성된 인공 신경망입니다.



이전 인공 신경망에 비해 합성 곱 층에 사용하는 필터의 개수를 많이 줄인 상태입니다.

이 신경망을 예제를 통해 살펴봅니다.

1. 다음과 같이 예제를 수정합니다.

523_1.py

```
17 model = nn.Sequential(
18         nn.Conv2d(1, 2, kernel_size=3, padding=1),    # padding='same'
19         nn.ReLU(),
20         nn.MaxPool2d(2),                              # 28x28 -> 14x14
21         nn.Flatten(),                                 # 2*14*14 = 392
22         nn.Linear(2 * 14 * 14, 10),                   # softmax 없음: CrossEntropyLoss가 내부에서 처리
23 ).to(device)
```

18 : nn.Conv2d를 이용하여 합성 곱 신경망 층을 생성합니다. 합성 곱 출력 층의 개수는 2개(즉, 출력 층의 깊이는 2), 사용할 필터의 크기는 3x3, 활성화 함수는 relu, padding을 'same'으로 설정하여 출력 이미지의 크기를 입력 이미지의 크기와 같게 합니다. 여기서 입력 이미지의 깊이는 1이기 때문에 3x3 필터의 깊이도 1이 되어야 하며, 합성 곱 출력 층의 개수가 2개(즉, 출력 층의 깊이는 2)이기 때문에 필터의 개수는 2개가 되어야 합니다. 즉, 깊이 1의 3x3 필터가 2개가 생성됩니다. 필터의 개수는 출력 층의 깊이와 같아야 합니다. 이 층에서 필요한 가중치 변수의 개수는 2*3*3*1=18개, 편향 변수의 개수는 2개가 되어 총 20개의 매개변수가 필요합니다. 편향의 개수는 필터의 개수와 같아야 합니다.

20 : nn.MaxPooling2d 클래스를 이용하여 풀링 층을 생성합니다. 풀링 층에 사용할 풀링 필터의 크기는 2x2입니다. 이전 합성 곱 층의 깊이가 2이므로 풀링 층의 깊이도 2가 되어야 하며, 이미지의 크기는 반으로 줄어 14x14가 됩니다.

21 : nn.Flatten 클래스를 이용하여 이전 풀링 층을 완전 연결 층의 입력 층으로 모양을 변경합니다. 이 과정에서 14*14*2=392개의 입력 노드로 변경됩니다.

2. ▶ 버튼을 눌러 프로그램을 실행시킵니다. 다음은 실행 결과 화면입니다.

```
Sequential(
  (0): Conv2d(1, 2, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1)) ❶
  (1): ReLU()
  (2): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  (3): Flatten(start_dim=1, end_dim=-1) ❷
  (4): Linear(in_features=392, out_features=10, bias=True) ❸
)
Total params: 3950 ❹
```

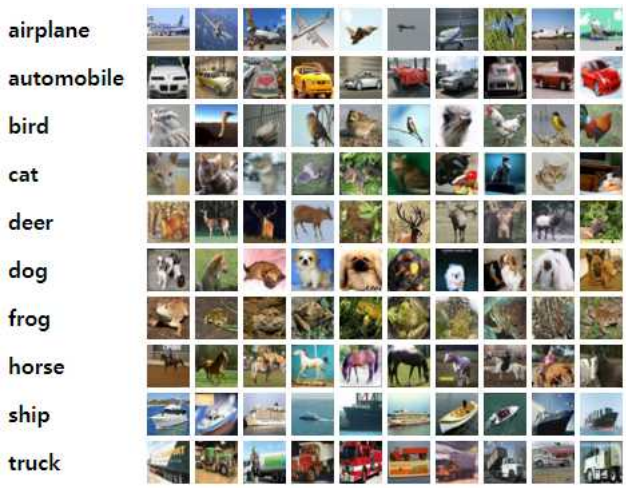
❶ 합성 곱 단계에서 사용되는 가중치는 3x3x1x2의 크기를 갖습니다. 편향은 가중치의 개수만큼 필요하여 2개가 됩니다. 그래서 합성 곱에 필요한 매개변수의 총 개수는 (3x3x1x2+2)=20이 됩니다. ❷ Flatten 층은 완전 연결 층의 입력 층으로 14x14x2=392개의 노드로 구성됩니다. ❸ 출력 층의 노드는 10개로 Flatten 층과 출력 층 연결에 필요한 가중치의 개수는 392x10이 됩니다. 편향은 가중치의 개수만큼 필요하여 10개가 됩니다. 그래서 Flatten 층과 출력 층 연결에 필요한 매개변수의 총 개수는 (392x10+10)=3930이 됩니다. ❹ 그래서 학습의 대상이 되는 매개변수의 총 개수는 3950이 됩니다.

```
Epoch 1/5 - loss: 0.6495 - accuracy: 0.7720 ❶
Epoch 2/5 - loss: 0.4864 - accuracy: 0.8288
Epoch 3/5 - loss: 0.4614 - accuracy: 0.8385
Epoch 4/5 - loss: 0.4502 - accuracy: 0.8425
Epoch 5/5 - loss: 0.4393 - accuracy: 0.8461 ❷
Test - loss: 0.4545 - accuracy: 0.8432 ❸
```

패션 MNIST 데이터의 학습 과정의 정확도는 ❶ 77.20%에서 시작해서 ❷ 84.61%로 끝납니다. 완전 연결 인공 신경망에 비해 3% 정도 정확도가 낮습니다. ❸ 학습이 끝난 후에, evalueate 함수로 시험 데이터를 예측한 결과는 84.32%로 학습 데이터의 예측 결과에 비해 정확도가 떨어진 상태입니다. 시험 데이터에 대한 예측 결과도 완전 연결 인공 신경망에 비해 1% 정도 정확도가 낮습니다.

04 cifar10 데이터 셋 학습하기

여기서는 CNN 신경망을 이용하여 CIFAR-10 데이터 셋을 학습시켜 봅니다.



<출처 : https://www.cs.toronto.edu/~kriz/cifar.html>

1. 다음과 같이 예제를 수정합니다.

524_1.py

```
10 train_ds = datasets.CIFAR10(root="./data", train=True, download=True, transform=transform)
11 test_ds = datasets.CIFAR10(root="./data", train=False, download=True, transform=transform)

17 model = nn.Sequential(
18         nn.Conv2d(3, 32, kernel_size=3, padding=1),    # padding='same'
19         nn.ReLU(),
20         nn.Conv2d(32, 64, kernel_size=3, padding=1),
21         nn.ReLU(),
22         nn.MaxPool2d(2),                               # 32x32 -> 16x16
23         nn.Flatten(),                                  # 64*16*16 = 16384
24         nn.Linear(64 * 16 * 16, 128),
25         nn.ReLU(),
26         nn.Linear(128, 10),                            # softmax 없음: CrossEntropyLoss가 내부에서 처리
27 ).to(device)
```

10, 11 : 데이터 셋을 CIFAR10으로 설정합니다.

18 : 입력 층을 CNN 신경망에 맞게 (3, 32)의 3차원 입력으로 맞춰줍니다.

26 : 출력 층의 개수를 10으로 맞춰줍니다.

2. 예제를 실행합니다. 다음은 실행 결과 화면입니다.

```
Epoch 1/5 - loss: 1.3621 - accuracy: 0.5105
Epoch 2/5 - loss: 0.9765 - accuracy: 0.6561
Epoch 3/5 - loss: 0.8033 - accuracy: 0.7180
Epoch 4/5 - loss: 0.6624 - accuracy: 0.7680
Epoch 5/5 - loss: 0.5380 ❶ accuracy: 0.8118 ❷
Test - loss: 0.9459 - accuracy: 0.6897 ❸
```

학습 마지막 단계에서 ❶ 오차는 0.5380, ❷ 학습 정확도는 0.8118이 나왔습니다. 검증 정확도는 ❸ 0.6897가 나왔습니다. 기본 신경망의 경우 오차는 1.5426, 학습 정확도는 0.4524, 검증 정확도는 0.4338이었습니다. CNN 신경망이 기본 신경망에 비해 학습 정확도는 1.78배 정도, 검증 정확도는 1.59배 정도 향상되었습니다.

05 cifar100 데이터 셋 학습하기

여기서는 CNN 신경망을 이용하여 CIFAR-100 데이터 셋을 학습시켜 봅니다.



<출처 : https://datasets.activeloop.ai/docs/ml/datasets/cifar-100-dataset/>

1. 다음과 같이 예제를 수정합니다.

525_1.py

```
10 train_ds = datasets.CIFAR100(root="./data", train=True, download=True, transform=transform)
11 test_ds = datasets.CIFAR100(root="./data", train=False, download=True, transform=transform)

17 model = nn.Sequential(
18         nn.Conv2d(3, 32, kernel_size=3, padding=1),    # padding='same'
19         nn.ReLU(),
20         nn.Conv2d(32, 64, kernel_size=3, padding=1),
21         nn.ReLU(),
22         nn.MaxPool2d(2),                              # 32x32 -> 16x16
23         nn.Flatten(),                                 # 64*16*16 = 12544
24         nn.Linear(64 * 16 * 16, 128),
25         nn.ReLU(),
26         nn.Linear(128, 100),                            # softmax 없음: CrossEntropyLoss가 내부에서 처
리
27 ).to(device)
```

10, 11 : 데이터 셋을 CIFAR100으로 설정합니다.

26 : 출력 층의 개수를 100으로 맞춰줍니다.

2. 예제를 실행합니다. 다음은 실행 결과 화면입니다.

```
Epoch 1/5 - loss: 3.4993 - accuracy: 0.1748
Epoch 2/5 - loss: 2.7680 - accuracy: 0.3086
Epoch 3/5 - loss: 2.4177 - accuracy: 0.3777
Epoch 4/5 - loss: 2.1281 - accuracy: 0.4400
Epoch 5/5 - loss: 1.8599 ❶ accuracy: 0.4997 ❷
Test - loss: 2.6854 - accuracy: 0.3535 ❸
```

학습 마지막 단계에서 ❶ 오차는 1.8599, ❷ 학습 정확도는 0.4997이 나왔습니다. 검증 정확도는 ❸ 0.3535가 나왔습니다. 기본 신경망의 경우 오차는 3.5702, 학습 정확도는 0.1561, 검증 정확도는 0.1481이었습니다. CNN 신경망이 기본 신경망에 비해 학습 정확도는 3.65배 정도, 검증 정확도는 2.39배 정도 향상되었습니다.